GPNB_v1 User Manual

Genomic Prediction Network Builder

Installation Operation Parameter Reference and Verification

Software version 1.0.0

This manual is intended for researchers with a background in genetics and breeding who wish to quickly and conveniently build deep learning genomic prediction models.

Contents

1 Software overview and scope 4
2 Installation, launch, and updates 4
2.1 Locate the application folder 4
2.2 First setup 5
2.3 Start and restart 5
2.4 Repair, move, or update 6
2.5 Select the environment explicitly 6
3 First complete example a small synthetic cohort 6
3.1 Train and inspect the result 7
3.2 Apply the model to new synthetic individuals 11
4 Prepare data and check matching 12
4.1 Start a data entry 13
4.2 Option A PLINK BED/BIM/FAM 13
4.3 Option B numeric matrix 13
4.4 Phenotypes and IDs 14
4.5 Check before fitting 14
4.6 Raw exports and fitted preprocessing 15
5 Build a prediction network 15
5.1 Choose the inputs before editing layers 16
5.2 Edit a network using layer cards 16
5.3 Sequential networks and branches 16
5.4 Available templates and components 17
5.5 Use a paper architecture deliberately 17
6 Choose an evaluation protocol 18
7 Set training parameters 19
7.1 Objective and optimizer settings 20
8 Configure training-fitted preprocessing 20
9 Search a small set of hyperparameters 21
10 Run checks, start training, and monitor progress 21
11 Train several traits separately or jointly 22
12 Optional staged training 23
13 Train a final model for new individuals 23
14 Read, export, and interpret results 24
14.1 Select the correct evaluation scope 24
14.2 Choose what to save 25
14.3 CSV field reference 26

15 Predict new individuals 26
16 Troubleshooting 28
17 Optional reproduce the small example with commands 29
18 Network components and all registered parameters 31
18.1 Read the layouts and ports 31
18.2 Parameter vocabulary 31
18.3 Dense layers, activations and dropout 32
18.4 Layout, slicing and marker grouping 33
18.5 One-dimensional convolution and pooling 34
18.6 Normalization 35
18.7 Recurrent layers 35
18.8 Genotype embeddings, tokens and masks 36
18.9 Attention and sequence mixing 37
18.10 Branches, gates and higher-level blocks 37
18.11 Locus graphs and individual references 38
18.12 Explicit two-dimensional grids 39
18.13 Autocorrelation and decomposition 40
18.14 Auxiliary reconstruction 40
18.15 Additional shape and interpretation rules 41

1 Software overview and scope

This guide describes the current `GPNB_v1` application, whose internal version is `1.0.0`. Screens and examples use synthetic individuals. The application is a Windows desktop workbench for continuous genomic prediction: configure data, construct a PyTorch regression network, evaluate it with a declared split, and apply a frozen model to query individuals. It is intended for researchers in genetics and breeding who prefer a graphical workflow.

Use this order on a first visit: install, run the small example, prepare your data, configure training, then interpret and export results. The command-line example is optional; the main workflow uses the interface.

- PLINK BED/BIM/FAM and explicitly configured diploid hard-call matrices; phenotype matching by individual identifiers.
- 89 network components, 30 generic templates, and eight documented paper-architecture adaptations, with input and shape checks.
- Training-fitted imputation, scaling and optional marker filtering; optional numeric input branches, kernel features and declared graph protocols.
- Fixed train/validation/test splits, five-fold and ten-fold CV, and leave-one-individual-out; internal parameter search and early stopping.
- Separate models for multiple traits or explicitly configured joint multi-target regression.
- Pearson correlation, RMSE, MAE and additional diagnostic metrics; prediction records, plots, CSV and offline HTML reports.
- Frozen model packages, complete experiment exports, and prediction of new individuals with optional scoring after prediction.

Implemented features include PLINK and explicit hard-call matrix inputs, ID-based phenotype matching, configurable networks, training-fitted preprocessing, fixed or cross-validation evaluation, parameter search, separate-trait tasks and joint multi-target regression, result reports, frozen model packages, and new-individual prediction. Supported components and protocols are described below; they are not all interchangeable.

The desktop is not a general-purpose VCF converter, a pedigree-analysis package, a classification tool, or an automatic estimator of breeding-value reliability. A predicted phenotype does not become an estimated breeding value merely because it was produced from markers. Predicting an unseen population or environment requires an appropriate evaluation design; a random individual split does not establish that performance. Paper architecture presets are documented adaptations, not claims of reproducing each author's published accuracy.

2 Installation, launch, and updates

2.1 Locate the application folder

The repository/application root is the folder containing `pyproject.toml`, `Setup_GPNB_v1.cmd`, `Start_GPNB_v1.cmd`, `src`, and `requirements`. Keep these files together. If you received `GPNB_v1.zip`, extract it completely to a writable folder before starting. Do not run the program from inside the ZIP archive. There is no need to install a separate Python distribution or use an environment manager yourself.

All commands in this guide are for **PowerShell opened in that application folder**, unless explicitly labelled otherwise. They are not Bash commands. In File Explorer, open the application folder, open a terminal there, and select a PowerShell tab.

Requirement	Current installer behavior
Operating system	Windows x64 (AMD64). The supplied installer does not support macOS, Linux, Windows ARM64, or WSL.
Installer shell	Windows PowerShell; the CMD entry files call `powershell.exe`.
Python	Local CPython 3.12.11, installed by setup. Package metadata requires Python 3.12.
CPU profile	PyTorch 2.7.1+cpu; at least 8 GiB free disk space before setup.
GPU profiles	`cuda`: PyTorch 2.7.1+cu126; `cuda118`: 2.7.1+cu118. Each requires at least 25 GiB free space and a compatible NVIDIA GPU/driver.
Other main dependencies	NumPy 1.26.4, PySide6 6.8.3, bed-reader 1.0.6, Matplotlib 3.10.1, psutil 7.0.0.
Network access	First setup downloads pinned Python and dependencies. Normal launch after successful installation works offline.

These free-space checks are installer thresholds, not estimates of RAM, GPU memory, or storage for a real experiment. The installer does not modify the system Python or NVIDIA driver. Model size, marker count, reference matrices, and data copies can require additional resources.

2.2 First setup

For the smallest demonstration, install CPU:

PowerShell

```
.\Setup_GPNB_v1_CPU.cmd -Language en_US
```

For an interactive choice of environment:

PowerShell

```
.\Setup_GPNB_v1.cmd -Language en_US
```

Choose `cpu`, `cuda`, or `cuda118` when prompted. Alternatively specify the profile directly:

PowerShell

```
.\Setup_GPNB_v1.cmd -Profile cuda -Language en_US
```

Setup creates `runtime`, `cache`, `tmp`, and `logs`. It downloads and verifies the installer components, creates an isolated environment, checks locked dependency hashes, installs the local application, and tests imports, BED reading, small CPU/GPU computations and Qt. A short health-check window may open and close. Successful setup ends with **`[5/5] Ready. Open Start_GPNB_v1.cmd; logs and diagnostics are saved.`** The selected environment has a `gpnb-ready.json` record, and `runtime/active-profile.json` records the active profile. A failed check does not mark the environment ready.

The recommendation uses installed driver information: current conservative Windows driver thresholds are 560.76 for `cuda` and 522.06 for `cuda118`. Passing this recommendation is not a GPU compatibility guarantee; the actual computation check must pass. Use CPU if a suitable GPU environment is unavailable. First-download duration has not been measured for this documentation.

2.3 Start and restart

PowerShell

```
.\Start_GPNB_v1.cmd -Language en_US
```

This starts the most recently configured profile. To select an installed CPU profile explicitly:

PowerShell

```
.\Start_GPNB_v1_CPU.cmd -Language en_US
```

Double-clicking the entry files without arguments defaults to Chinese. The language selector at the top of the window also switches between Chinese and English. Use `-Language en_US` on each English launch; the

launcher does not restore the language saved by setup. The CPU shortcut chooses the runtime, but does not change a loaded project's training device. Review **Device** and **Prediction device** in the application.

For Windows **Command Prompt (CMD)** only, the equivalent English launch from the application folder is:

CMD

```
Start_GPNB_v1.cmd -Language en_US
```

2.4 Repair, move, or update

If setup fails, read the newest `logs/setup-*.log`, its `.status.json` companion, and any `logs/doctor-*.json`. Restore connectivity or free space as appropriate, then rerun the same setup command. For an installed CPU environment that needs repair:

PowerShell

```
.\Setup_GPNB_v1.cmd -Profile cpu -Repair -Language en_US
```

Replace `cpu` with the profile being repaired. Close application workers before repair. `-Offline` is only for setup when the full required installer/runtime/dependency cache already exists; the downloadable ZIP itself is not an offline dependency bundle. Do not add `-Offline` to the start command.

There is no in-app updater or verified public update URL in this package. When the author supplies a new release, extract it to a separate writable folder, run that copy's setup, and reopen a copy of your project. Keep prior projects, complete experiment directories and model packages. Recheck data, splits and settings in the new copy before running an experiment. Do not merge virtual environments from different copies. After moving an installation, rerun setup from its new location; launch checks bind the environment to its application directory. Compatibility with a future unreleased version is not promised.

2.5 Select the environment explicitly

The following commands preserve the explicit alternatives listed in the standalone installation guide. Run them in **PowerShell opened in the extracted GPNB_v1 application folder**, which contains `Setup_GPNB_v1.cmd`, `Start_GPNB_v1.cmd`, and `pyproject.toml`. Do not run them from inside the ZIP archive.

As an alternative to the CPU shortcut, select CPU through the general setup entry point:

PowerShell

```
.\Setup_GPNB_v1.cmd -Profile cpu -Language en_US
```

To install the CUDA 11.8 environment explicitly:

PowerShell

```
.\Setup_GPNB_v1.cmd -Profile cuda118 -Language en_US
```

To start an already installed CPU environment by naming its profile:

PowerShell

```
.\Start_GPNB_v1.cmd -Profile cpu -Language en_US
```

These commands use the existing entry points and do not introduce additional installation modes. GPU profiles still require a compatible NVIDIA GPU and driver. Selecting the CPU launcher profile chooses the runtime; it does not change the training device saved in a project.

3 First complete example a small synthetic cohort

The supplied example has 65 synthetic individuals, 32 synthetic markers, and a continuous target named `trait`. Two target values are intentionally missing. It also contains eight independently simulated query individuals. No research or private individual data are included.

The example uses CPU, three epochs, patience 3, batch size 16, and one thread to keep the demonstration small. These are **example overrides**, not new-project defaults or recommended settings for a research study.

The ordinary new-project defaults include 100 epochs, patience 20, batch size 64 and two threads. The example does not enable parameter search or CV.

3.1 Train and inspect the result

1. Launch the CPU interface with the command above. The six tabs are **Data and project**, **Network builder**, **Training settings**, **Run monitor**, **Training results**, and **New individual prediction**.
2. Click **Open** at the top and select `examples/quickstart/project.json`. The project name is **Synthetic quick start**, the dataset identifier is `synthetic_quickstart`, and the target is `trait`. Paths inside the project are relative to the project file and are resolved when it opens.
3. Click **Save as** to save a working project under a new filename, for example `projects/quickstart-demo.json`. Use the directory selector for **Run output directory** if you want a different writable destination. The example initially resolves its output root to the application's `runs` folder.
4. On **Data and project**, scroll down and click **Check data and matching**. Expect 65 genotype records and 32 markers, with 63 finite matched labels and two missing labels. This GUI check does not count all genotype missing cells; the full import does.
5. Open **Network builder**. Keep the supplied `Linear 64 → ReLU → Linear 16 → ReLU → Linear 1` network. No layer editing is required for this example.
6. Open **Training settings**. Confirm **Fixed split**, CPU and the three-epoch example settings. Click **Generate / preview actual split**. This exact example produces **44 training, 10 validation, and 9 test individuals**. The two missing targets are excluded from this single-trait fit.
7. Scroll down and click **Check before running**. Wait for the checklist to pass. The check includes a bounded numerical probe, not a completed evaluation experiment. **Start training** becomes available after a valid check; editing relevant inputs or settings requires a new check.
8. Click **Start training**. Inspect **Run monitor** while the worker runs. On success, the application opens **Training results**. The GUI creates a new directory under `runs/experiments/<timestamp>_<suffix>`; it does not use the fixed command-line example names below.
9. In **3. Prediction results**, keep **Out-of-sample test / OOF**. For this fixed split the view contains nine held-out test observations; it is not a pooled CV result. Review the observed–predicted plot, individual errors and training/validation curves.
10. Under **4. Save and apply**, choose **Save location**, then click **Export results** for figures/CSV/HTML or **Save model** for a frozen prediction package. Each operation creates its own uniquely named output directory. Use **Open result directory** to locate the displayed experiment.

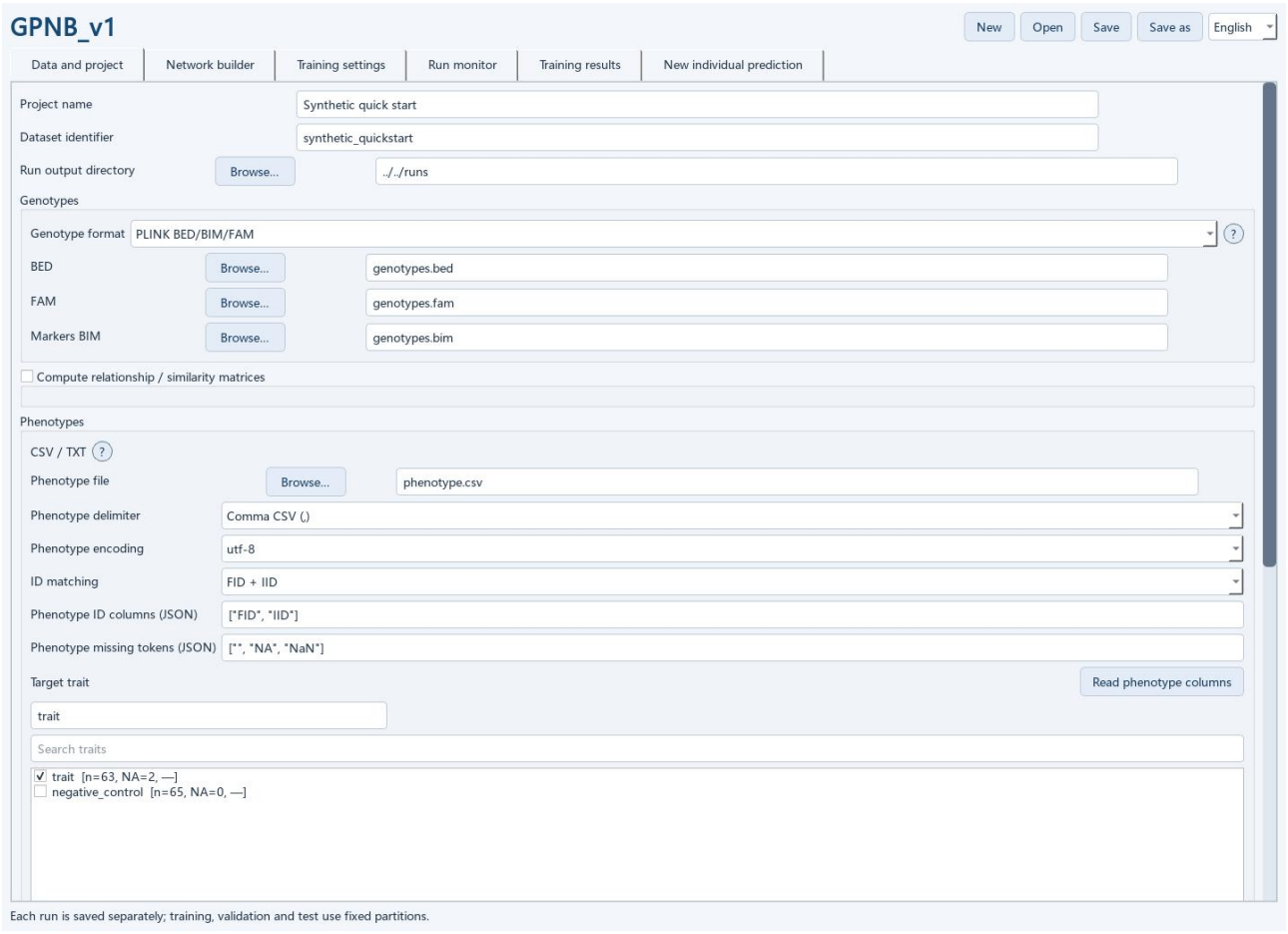


Figure 1 Data and project with synthetic inputs



Figure 2 Data inspection and sample matching results

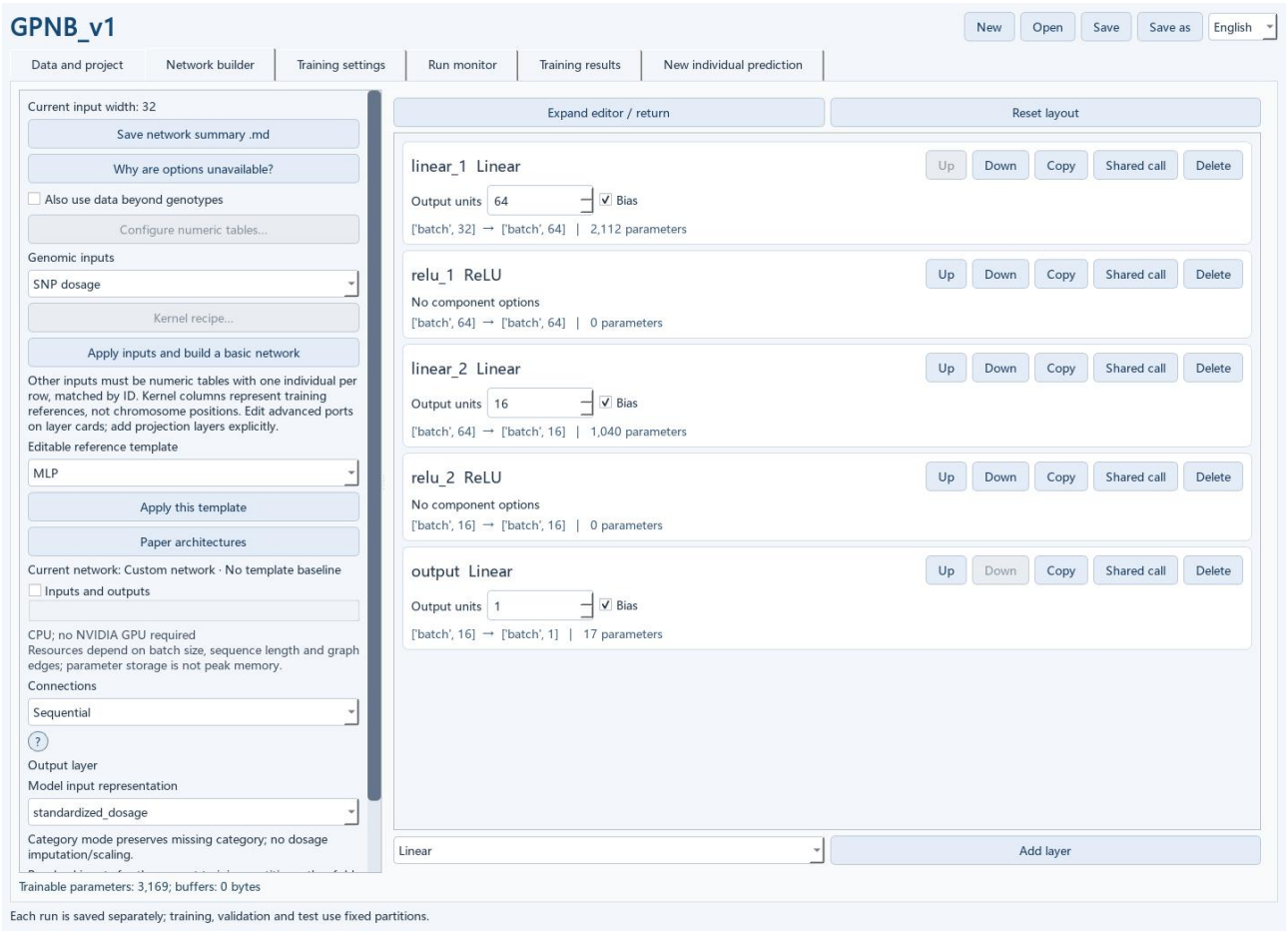


Figure 3 Real network-builder controls

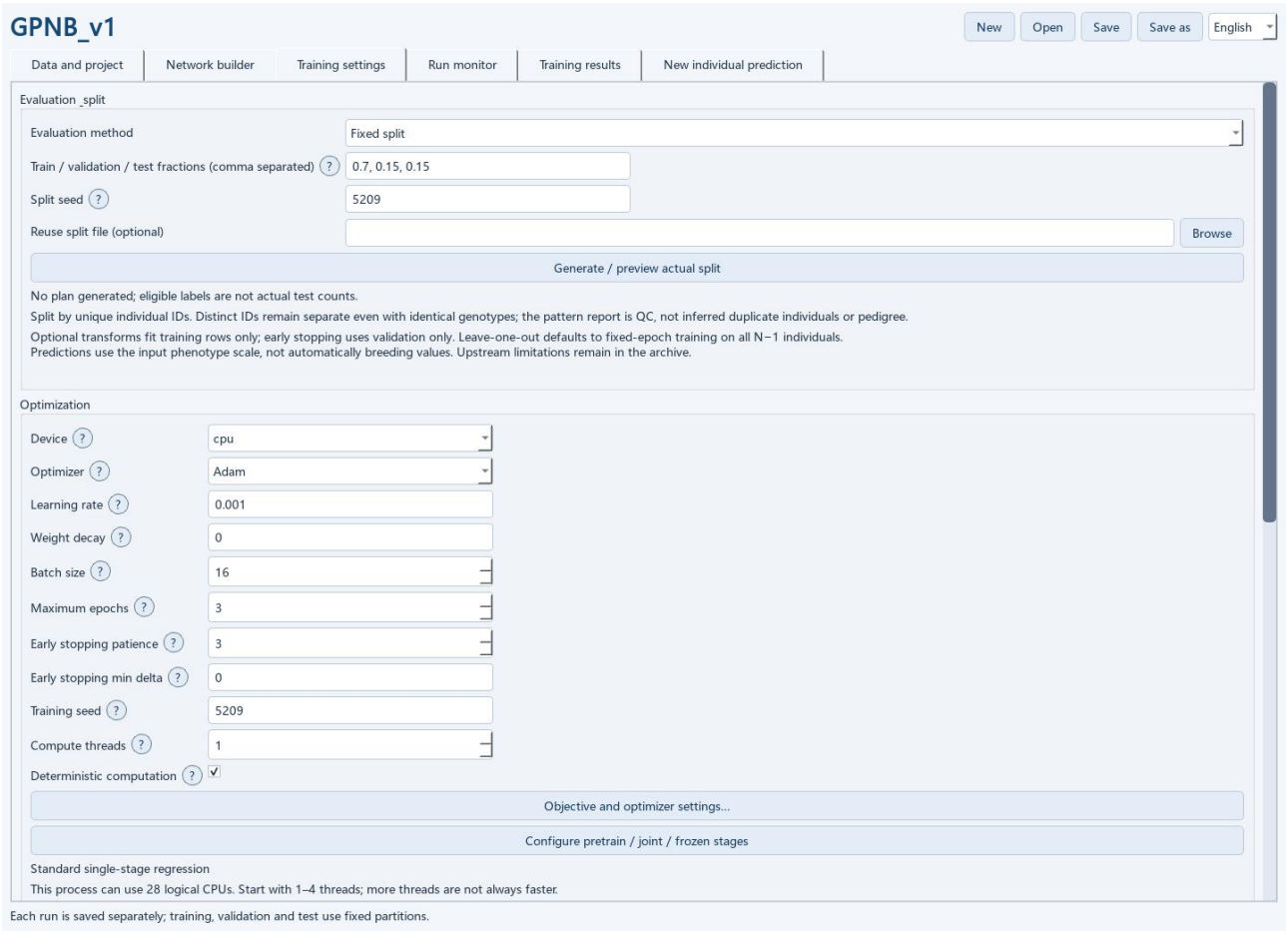


Figure 4 Example training configuration

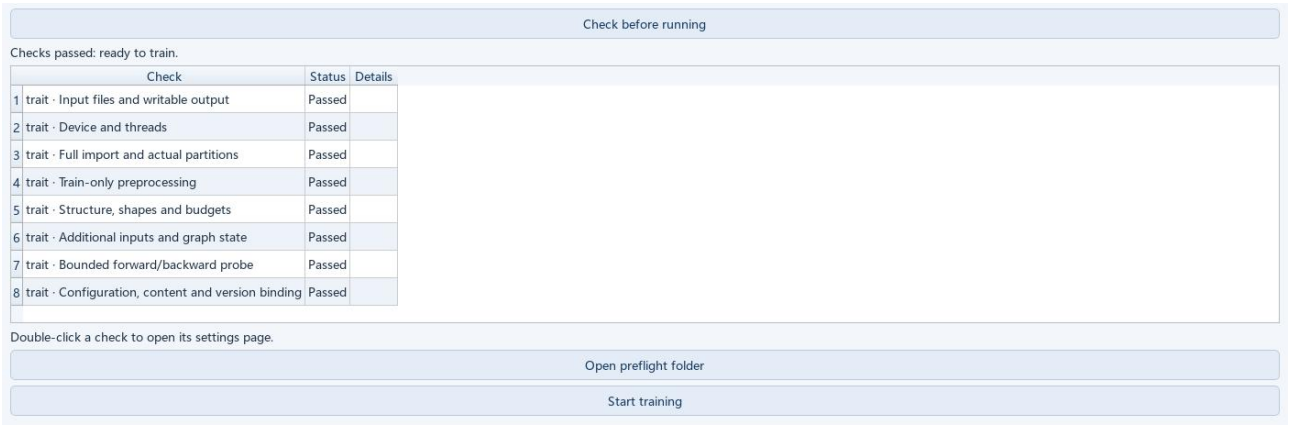


Figure 5 Preflight checks and training readiness



Figure 6 Actual synthetic test results and learning curves

3.2 Apply the model to new synthetic individuals

1. Open **New individual prediction**. Use **Browse…** to select the complete model directory created by **Save model**, then click **Load and check model**. A completed experiment is also accepted. Do not select only `weights.pt`.
2. Use the file selectors to choose `examples/quickstart/query/genotypes.bed` and its matching `.bim` and `.fam`. Review any automatically selected companion paths. For GUI prediction/result paths use the selectors rather than manually typing relative paths; see the path limitation.
3. Set **Query cohort name** to `synthetic_query`. For this particular independently generated example, select **Independently confirmed new individuals** and enter the basis: `Independent synthetic cohort with disjoint QRY identifiers.` Do not copy that assertion for real material without independent identity evidence.
4. Leave **Provide observed phenotypes for scoring** unchecked for prediction without labels. Select a writable **Output root**, CPU, and **Keep input order**.
5. Click **Check and predict**. Expect eight predictions. The result directory is under `runs/predictions/<timestamp>_<suffix>`. `predictions.csv` contains values and identities; `report.json`

records the prediction operation. A prediction without observed phenotypes has no measured prediction accuracy.

6. Optionally repeat into a new output using **Provide observed phenotypes for scoring** and `query/phenotype.csv`, with comma delimiter, UTF-8, FID/IID and trait `trait`. Those observations are used only after prediction for scoring. The CLI version with optional scoring was also verified.

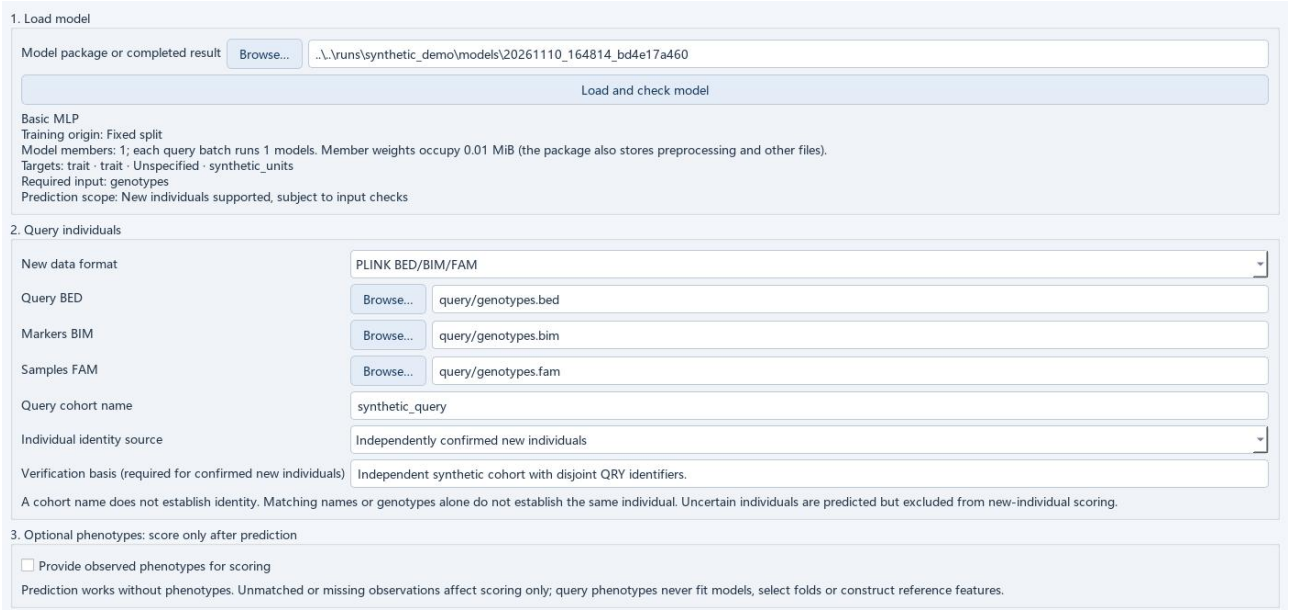
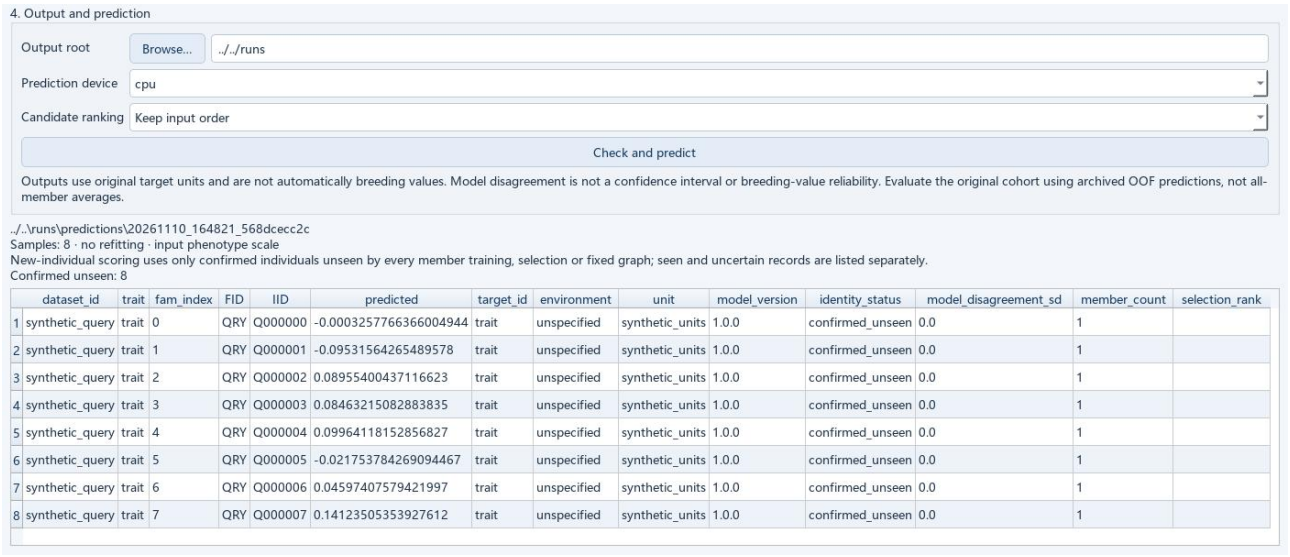


Figure 7 New-individual prediction controls



	dataset_id	trait	fam_index	FID	IID	predicted	target_id	environment	unit	model_version	identity_status	model_disagreement_sd	member_count	selection_rank
1	synthetic_query	trait	0	QRY	Q000000	-0.0003257766366004944	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	
2	synthetic_query	trait	1	QRY	Q000001	-0.09531564265489578	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	
3	synthetic_query	trait	2	QRY	Q000002	0.08955400437116623	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	
4	synthetic_query	trait	3	QRY	Q000003	0.08463215082883835	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	
5	synthetic_query	trait	4	QRY	Q000004	0.09964118152856827	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	
6	synthetic_query	trait	5	QRY	Q000005	-0.021753784269094467	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	
7	synthetic_query	trait	6	QRY	Q000006	0.04597407579421997	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	
8	synthetic_query	trait	7	QRY	Q000007	0.14123505353927612	trait	unspecified	synthetic_units	1.0.0	confirmed_unseen	0.0	1	

Figure 8 Predictions for eight independently simulated individuals

For the equivalent CSV input, open `examples/quickstart/project_matrix.json`; its `expected_dosage.csv` encodes the same hard calls and uses `-1` as an explicitly declared missing token under **0/1/2** coding. The example README explains why it has no PLINK source manifest. The decoded matrix was verified to match the BED data exactly.

4 Prepare data and check matching

GPNB_v1 accepts PLINK BED/BIM/FAM files or an explicitly configured hard-call text matrix. Training phenotypes come from a separate CSV, TSV, or whitespace-delimited text table. Excel workbooks, VCF files, continuous dosage matrices, and transposed marker-by-individual tables are not direct inputs to these screens. Convert them outside GPNB and retain the conversion and allele-coding records.

The supplied examples/quickstart folder contains only synthetic data. No research datasets or development test suite are bundled. The short tables below illustrate parser conventions; they are not a training benchmark.

4.1 Start a data entry

Open **Data and project**. Enter **Project name**, **Dataset identifier**, and **Run output directory**. Dataset identifiers label records; they do not establish that two cohorts contain the same individuals. Choose a separate output directory with sufficient disk space, then select the genotype format and phenotype file.

4.2 Option A PLINK BED/BIM/FAM

Select **PLINK BED/BIM/FAM**, then supply **BED**, **FAM**, and **Markers BIM** from the same export.

File	Required interpretation
`.bed`	Variant-major/SNP-major binary BED. The header and file size must agree with the numbers of individuals and markers in FAM/BIM.
`.bim`	Six whitespace-separated columns: chromosome, marker ID, genetic position in cM, physical position in bp, A1, A2. Marker IDs must be unique; bp must be a nonnegative integer; A1 and A2 cannot be identical.
`.fam`	Six whitespace-separated columns: FID, IID, father, mother, sex, phenotype. FID/IID pairs must be unique. The selected target is read from the separate phenotype table, not automatically from the FAM phenotype column.

Decoded calls count BIM A1 copies: 0, 1, or 2. GPNB does not infer ploidy. FID is an identifier field, not an automatic declaration of a biological family. Keep BIM and FAM attached to their original BED; do not independently reorder these files.

4.3 Option B numeric matrix

Select **Numeric matrix CSV / TXT**. Each row is one individual; each non-ID column is one uniquely named marker:

CSV

```
FID,IID,rs1,rs2,rs3
F01,0001,0,1,NA
F01,0002,2,0,1
```

For an IID-only source:

CSV

```
IID,rs1,rs2,rs3
0001,0,1,NA
0002,2,0,1
```

Set the fields explicitly:

Field	Initial setting and accepted choices
Matrix delimiter	Comma CSV initially; Tab and Whitespace TXT are available.
Text encoding	`utf-8` initially; also `utf-8-sig` and `gb18030`. Use `utf-8-sig` when a UTF-8 byte-order mark is present.
Genotype coding	`0 / 1 / 2` initially; alternatively `-1 / 0 / 1`, which maps to internal 0/1/2 by adding 1.
ID columns (IID alone or FID, IID; JSON)	`["FID", "IID"]` initially. Use `["IID"]` for an IID-only matrix, or supply your actual column names in the same semantic order.
Missing tokens (JSON)	`["", "NA", "NaN"]` initially; matching is exact and case-sensitive.

Field	Initial setting and accepted choices
Allele basis	**Count BIM A1 (BIM required)** initially; alternatively **Use source coding declaration (without BIM)**.

Accepted genotype tokens are exact strings. For example, `1` is valid under 0/1/2 coding; `1.0`, `0.7`, an unlisted missing token, or a call with surrounding spaces is rejected. Continuous dosage is never rounded. Under −1/0/1 coding, `-1` is an observed genotype, not a missing value. For whitespace-delimited input remove the empty-string missing token and use explicit tokens such as `["NA", "NaN"]`; spaces cannot represent an empty cell reliably.

With BIM, marker column names and order must match that matrix's BIM exactly and values must count A1. Without BIM, enter a meaningful **Coding source declaration**, such as a documented panel/vendor encoding identifier. Prediction requires the same declaration. This declaration does not independently verify biological allele direction, and genomic position information is unavailable. Do not use no-BIM mode merely to bypass known allele inconsistencies.

Use **Preview first 8 rows** to check delimiter, column names, and coding. This bounded preview does not replace the data check or full import.

4.4 Phenotypes and IDs

Example phenotype table:

CSV

```
FID,IID,height,yield
F01,0001,12.4,3.8
F01,0002,NA,4.1
```

New projects default to comma CSV, `utf-8-sig`, **FID + IID**, ID columns `["FID", "IID"]`, and missing tokens `["", "NA", "NaN"]`. This phenotype encoding default differs from the initial numeric-matrix encoding. Select **IID** and `["IID"]` when using an IID-only genotype matrix. Other column names are allowed when configured explicitly.

IDs are matched as strings, never by row position. Leading zeros matter: `0001` and `1` are different IDs. IDs must be nonempty and have no surrounding whitespace. Duplicate phenotype keys are rejected. IID-only matching also requires IID to be unique across all genotype rows, even if the original FID/IID pairs were unique. When preparing files in a spreadsheet editor, preserve ID columns as text.

Click **Read phenotype columns**, select **Target trait**, and confirm that numeric columns are actual targets rather than covariates. Missing values are excluded from supervised evaluation; they are not imputed as observed truth. `Inf`, unlisted `nan` tokens, and nonnumeric target values are invalid rather than legitimate missing observations. A new project's primary trait placeholder is `trait`; replace it with a real column.

For several selected traits, **Separate training (one model/trait)** creates separate tasks; **Joint training (shared multi-output model)** uses explicitly configured targets, units, and weights. Click **Apply selected traits and task mode**. The selector's `n` and `NA` counts describe the phenotype table; actual matched and partitioned counts come from the data check and pre-run preview. Different traits can have different usable individuals.

4.5 Check before fitting

Click **Check data and matching**. Review **Summary**, **Individuals**, and **Traits**, optionally filtering to **Unmatched or missing only**.

Status	Meaning
`matched_finite_label`	A matched ID has a finite selected-trait value.

Status	Meaning
`matched_missing_label`	The matched trait uses a configured missing token.
`matched_invalid_label`	The matched selected-trait value cannot be parsed as a finite observation.
`genotype_only`	No phenotype ID matches this genotype individual.
`phenotype_only`	The phenotype ID has no genotype match.

For joint tasks, eligibility is assessed across the configured targets; the audit's primary-trait display alone does not describe every target's missing-value mask.

The GUI check validates files, IDs, and phenotypes; it does **not** perform the complete genotype-missingness scan. Full training import performs that scan. A blank missing-genotype count in the preliminary audit means “not scanned”, not zero missingness. Matrix parsing may already reject malformed tokens while reading individuals; this still does not make the preliminary report a complete missingness scan.

The on-screen tables show at most 2,000 rows. Export audit tables

copies `samples.csv`, `phenotype_only.csv`, `invalid_traits.csv`, `trait_statistics.csv`, and `audit.json` into a new folder. **Open latest check folder** opens the full check directory, which also contains marker and phenotype-order records. Descriptive trait SD uses finite ID-matched observations and denominator n−1; it is not the cross-validation SD.

Optional **Advanced: provenance and duplicate samples** inputs record upstream processing or genotype-equivalence groups. Equivalence groups are neither pedigrees nor populations and only constrain a split when that policy is explicitly selected. Their CSV header is:

CSV

```
FID,IID,genotype_equivalence_group,group_size,basis
```

4.6 Raw exports and fitted preprocessing

Export raw 012 matrix creates `calls.txt` with tab separation and `NA` missing tokens, `matrix_config.json`, `encoding.json`, `report.json`, and a `markers.bim` companion when BIM-based. The first columns are normally FID/IID, or IID for an IID-only source; remaining headers are marker IDs. If a marker already uses an ID-column name, safe alternative ID names are recorded in the configuration. This export applies no imputation, filtering, or standardization.

By contrast, the default training preprocessing uses training-row mean imputation and genotype standardization; target standardization is initially off, and MAF/missing-rate thresholds are initially unset. All-missing training markers are dropped by default. Each fold fits its own preprocessing state. Predictions reuse saved states rather than fitting a new transform on the query cohort. A record of upstream whole-cohort processing cannot be undone by a later training-only transform.

An independent **Configure and compute GRM / kernel matrix…** export is descriptive. To train with a kernel, select the training-fitted kernel recipe in **Network builder**. Extra numeric branches also require their own explicit ID-matched input tables; query prediction asks for a fresh table for every required branch.

5 Build a prediction network

Open **Network builder** after configuring the genotype and phenotype files. A new project starts with a small multilayer perceptron: `Linear 64 → ReLU → Linear 16 → ReLU → Linear 1`. The last unit predicts one continuous phenotype. These are starting settings, not parameters optimized for your population or trait.

5.1 Choose the inputs before editing layers

Under **Genomic inputs**, choose **SNP dosage**, **GRM / kernel features only**, or **SNP + GRM / kernel features**. For a kernel input, open **Kernel recipe…** first. If you also have covariates or other omics measurements, select **Also use data beyond genotypes**, then **Configure numeric tables…**. These tables must contain one individual per row and numeric feature columns matched by ID.

Click **Apply inputs and build a basic network** to create a network for the declared inputs. This button replaces the network; save the project under another name first if you want to keep the previous architecture. Selecting an input mode alone does not silently rebuild a network.

Kernel features describe similarity to training reference individuals. Their columns are not SNP positions. The builder therefore restricts marker-specific components and templates in kernel-only mode. It also prevents a single-input template from silently discarding an existing additional input table.

5.2 Edit a network using layer cards

1. Select a name under **Editable reference template**, then click **Apply this template**, or keep the initial MLP.
2. Read the layer cards from top to bottom. Change a layer's parameters directly; for `Linear`, **Output units** controls its width and **Bias** controls the intercept term.
3. Select a component beside **Add layer**. New hidden layers are inserted before the regression output. Use the card buttons to move, copy, share, or delete a layer.
4. **Copy** creates a separate set of trainable parameters. **Shared call** reuses another layer's parameters and saved state; changing the owning layer's parameters also updates its shared calls.
5. Open **Shapes and summary** to inspect inferred input/output sizes and parameter storage. Use **Save network summary .md** to save a readable architecture description.

The input feature count is obtained from the data, and the retained count can change after training-fitted filtering. It is not a number to invent in the interface. Before dimensions are known, some shapes remain unresolved. **Expand editor / return** enlarges the card area; **Reset layout** restores its layout. The current interface uses cards and the shape summary; it does not provide an interactive network diagram view.

The component menu is filtered by the current input type and insertion point. If an option is missing, click **Why are options unavailable?**. A hidden option can require a different input layout or an explicit graph configuration; it does not mean the component is absent from the program.

5.3 Sequential networks and branches

In **Connections → Sequential**, layers form one chain. In **Branches**, each card exposes its input ports, and **Output layer** identifies the prediction output. Use branches for residual connections, multiple input tables, masks, shared calls, and attention with separate query/context inputs. Components with several input ports automatically require branch mode.

GPNB_v1 checks connections and dimensions. For example, `Add` requires compatible tensors, and a recurrent layer expects a sequence rather than a flat marker vector. It does not insert projection layers automatically. Add the required `Linear`, layout conversion, pooling, or flattening component explicitly. Returning from branches to a sequential network is allowed only when the existing connections already form a simple chain; branches are not discarded automatically.

For a single trait, preserve a linear output with one unit. A multi-target network must have one output unit per declared target. The current model contract also allows one explicit `Dropout` immediately after the linear output; it is disabled during prediction. An arbitrary activation after the regression output is not accepted.

5.4 Available templates and components

There are **89 registered components**, with editable parameters described in the component reference. Components include dense layers, activations, normalization, convolution, pooling, recurrent networks, tokenization, attention, graph operations, input fusion, and reconstruction. A component is an individual building block; a template is a connected starting network containing several components.

The builder contains these **30 generic template choices**. Availability depends on the declared inputs.

Purpose	Template names	What to check
Dense and residual regression	MLP, Residual, GatedAdd, GatedMLP, DenseMoE	Widths, regularization, and branch connections
Marker-order convolution	CNN, Local, SE, TCN	Retained marker count, kernel sizes, pooling, and memory
Recurrent processing	RNN, LSTM, GRU	Sequence layout and readout; the generic templates pool marker positions before the recurrent layer
Tokens and attention	Tokens, SelfAttention, Transformer, CrossAttention, Mixer, AttentionPool	Token count, embedding width, masks, and attention memory limits
Genotype categories	Categorical	Uses explicit genotype/missing categories rather than continuous-dose imputation and scaling
Locus graphs	GCN, GraphSAGE, GIN, GAT	Configure a locus graph before applying the template
Multiple inputs and reference features	MultiInput, ReferenceGraph, KernelMLP, SNP+Kernel, FeatureAffine	Required tables, kernel mode, or individual reference protocol must be explicit
Grid and auxiliary reconstruction	GridCNN, Autoencoder	Grid layout is a user arrangement; Autoencoder adds a reconstruction head and explicit training stages

Convolution windows, marker chunks, token positions, and the generic grid follow retained marker/file order. They do not automatically infer chromosomes, physical distance, LD blocks, or biological images. Locus graphs and individual reference graphs are different inputs: configure them through **Import locus graph / inspect mapping** and **Individual reference protocol**, respectively. The fixed-cohort reference protocol does not support arbitrary previously unknown individuals; use the inductive training-reference protocol when that is the intended prediction task.

5.5 Use a paper architecture deliberately

Click **Paper architectures** to inspect the available adaptations: **DeepGS, DeepGP, DLGWAS, deepGBLUP, WheatGP, Cropformer, DNNGP, and GPformer**. Read the architecture, input requirements, and parameter preview before applying it.

The dialog provides **Apply verified literature settings** and **Architecture only; retain training settings**. Click **Apply preview** to apply the displayed changes; **Undo application** restores the prior configuration while no subsequent project edits would be discarded. If a staged network is open, the dialog requires an explicit choice about removing its stages. Project preprocessing, split, and search settings remain in place. An existing learning-rate schedule can therefore remain unless the preview explicitly changes it.

Preset	Training fields applied by the preset
DeepGS	SGD; learning rate 0.01; momentum 0.5; weight decay 0.00001; batch size 30; maximum epochs 6000; training seed 0; MSE

Preset	Training fields applied by the preset
DeepGP	MSE only
DLGWAS	Adam; learning rate 0.001; batch size 250; maximum epochs 1000; MSE
deepGBLUP	No additional training scalar settings
WheatGP	Adam; learning rate 0.005; weight decay 0.0001; batch size 64; maximum epochs 300; MSE; gradient norm cap 1; StepLR every 90 epochs, multiplier 0.1
Cropformer	Adam; learning rate 0.001; weight decay 0; batch size 32; maximum epochs 100; MSE
DNNGP	No additional training scalar settings; PCA and multi-omics concatenation are not applied automatically
GPformer	Adam; learning rate 0.0001; weight decay 0.00001; batch size 32; focal MAE; $\beta = 0.5$ and $\gamma = 1$

These are editable native PyTorch adaptations. In particular, DNNGP and GPformer are paper-based architecture mappings. The deepGBLUP option implements a neural subgraph rather than full GBLUP variance/joint estimation. Cropformer retains the mapped single-token attention behavior. The GPformer option does not include a GWAS-guided KGM branch.

6 Choose an evaluation protocol

Open **Training settings → Evaluation_split**. Decide which individuals should be withheld before choosing model settings. The current default split unit is the unique individual ID. Identical genotype patterns with distinct IDs remain distinct individuals; the genotype-pattern QC report does not infer duplicate identities or pedigree. Ordinary random individual splits do not establish performance in a new family, population, generation, or environment.

Evaluation method	Current behavior	Controls to review
Fixed split	One train/validation/test split; default fractions 0.70/0.15/0.15	Fractions, split seed, optional saved split
Five-fold cross-validation	Five outer test folds; an inner validation subset is taken from each outer training set	Outer/inner split seed; validation fraction within the outer training set, default 0.20
Ten-fold cross-validation	Ten outer test folds, each with its own inner validation subset	Same controls as five-fold CV
Leave one individual out	One individual is withheld per fold; the default trains on all remaining eligible individuals for fixed epochs	Training policy; split preview; acknowledgement of the actual number of fits

For five-fold CV with a 0.20 inner-validation fraction, approximately 64% of eligible individuals train the model, 16% select the checkpoint, and 20% form the outer test set in each fold. These are proportions for understanding the protocol; actual integer counts come from the generated plan. Ten-fold CV gives approximately 72%/18%/10% with the same inner fraction.

Click **Generate / preview actual split**. Inspect the train, validation, and test counts, including trait-specific missing labels. A count of nonmissing phenotypes is not automatically the number of outer-test observations. For leave-one-out, acknowledge the displayed outer-fit count before preflight and training. The interface also retains an explicit inner-holdout leave-out policy for existing workflows; that policy trains on fewer than

N−1 individuals and uses validation early stopping. It is a different protocol from the default full-training leave-one-out option.

Use **Reuse split file (optional)** for a compatible fixed split or **Reuse cv_plan.json** for a compatible CV plan. Reuse preserves declared membership; changing the data or scientific settings may invalidate the plan. For a comparison across models, retain the same eligible cohort and the same split plan. Repeating a protocol with different seeds requires separate configurations/runs; selecting five-fold CV does not automatically mean three repetitions or 15 folds.

7 Set training parameters

The following values are the **new-project defaults**, before a saved project or paper preset changes them. Training and split seeds are separate controls.

GUI field	Default	Meaning and allowed choices
Device	cpu	`cpu` or `cuda`. CUDA requires an installed, usable NVIDIA environment; unavailable CUDA does not silently become CPU.
Optimizer	Adam	Adam or SGD.
Learning rate	0.001	Positive update step size. Compare alternatives using internal validation.
Weight decay	0	Nonnegative optimizer weight decay; not an arbitrary paper's layer-specific regularization.
Batch size	64	Individuals per update, at least 1. Lower values can reduce memory use.
Maximum epochs	100	Upper limit for a standard fit. Early stopping may finish earlier.
Early stopping patience	20	Consecutive epochs without the required validation-loss improvement.
Early stopping min delta	0	Nonnegative improvement threshold for resetting patience.
Training seed	5209	Controls model/training randomness; integer from 0 to 4294967295.
Compute threads	2	Positive CPU thread count; the GUI limits selection to available process CPU affinity.
Deterministic computation	On	Requests supported deterministic operations; it does not establish equality across different hardware, libraries, or model implementations.
Loss, in advanced settings	mse	`mse`, `mae`, or `focal_mae`.
Split seed	5209	Seed for a fixed split; CV has its own outer/inner split seed, also initially 5209.

With validation-based training, GPNB_v1 restores the weights with the **smallest recorded validation objective**. `min_delta` controls the patience counter, not which minimum checkpoint is retained. With fixed-epoch training there is no validation-based stopping and the final epoch is retained. Outer-test results do not select the checkpoint.

7.1 Objective and optimizer settings

Click **Objective and optimizer settings…** for the following options:

Option	Default when not configured	Meaning
Loss	mse	MSE squares errors; MAE uses absolute errors.
SGD momentum	0	Available for SGD; accepted range 0–1.
Focal β / Focal γ	0.5 / 1	For focal MAE: `tanh(β × absolute error)^γ × absolute error`. β must be positive and γ is 1–100.
Learning-rate schedule	None	ExponentialLR or StepLR can be enabled explicitly.
Decay multiplier	Dialog starts at 0.95	Used only when a schedule is enabled; accepted values are greater than 0 and at most 1. This is an editable example, not a universal paper setting.
Decay every epochs	Dialog starts at 90	Positive interval used only by StepLR.
Gradient norm cap	Blank/off	A positive cap enables gradient clipping.

Schedulers update after training epochs. The objective uses the configured training-target scale: if target standardization is enabled, its loss values are on that standardized scale. Reported predictions and regression-error metrics are returned to the input phenotype scale. Pearson correlation and RMSE describe different aspects of prediction; neither is automatically an estimate of breeding-value accuracy.

8 Configure training-fitted preprocessing

The **Train-fitted preprocessing** section is enabled for a new project. Fit statistics are calculated from the applicable training partition and then reused for validation, outer-test, and new individuals. Each CV fold fits its own preprocessing. Internal search partitions and final all-data refitting have their own fitting boundaries.

Field	New-project default	Options and interpretation
Missing genotypes	mean	`mean`, `none`, `mode`, `median`, or `knn`. Mean and median can produce fractional dosage. Mode ties use the smaller code.
All-missing training markers	drop	`drop`, `error`, or `zero_with_warning`; mode/median cannot estimate an entirely missing retained training marker.
Standardize genotypes	On	Uses centers/scales fitted after training-set imputation. Constant features use scale 1.
Standardize target	Off	When enabled, uses training mean and population SD; output predictions are inverse transformed.
Minimum train MAF	Blank/off	Optional value between 0 and 0.5.
Maximum train marker missingness	Blank/off	Optional value between 0 and 1.

Turning preprocessing off sets imputation to `none`, disables genotype and target scaling, and clears marker filters. Format conversion and ID matching still occur. Missing continuous dosage then blocks computation; category-based inputs retain an explicit missing category. Additional numeric tables keep their own configured preprocessing. Re-enabling the switch does not silently restore all previously hidden transformations: review the visible fields again.

For KNN, click **Training-reference KNN options…**. Defaults are 5 neighbors, at least 1 jointly observed marker, uniform weighting, and continuous-mean output. Inverse-distance weights and hard-call voting are alternatives. Defaults cap references at 20,000 individuals, reference storage at 268,435,456 bytes, and working storage at 67,108,864 bytes. Only frozen training references are used; prediction queries never become references for one another. A marker-specific fallback is saved for cases without usable neighbors. Large training cohorts can hit these explicit limits even when the computer has free memory.

9 Search a small set of hyperparameters

Enable **Enable hyperparameter search**, then open **Configure random / grid search**. Select a parameter, enter explicit comma-separated candidate values, and click **Save settings**. For example, learning rates `0.0001, 0.001, 0.01` and batch sizes `16, 32` form six combinations.

Search setting	Initial value	Behavior
Method	random	Samples combinations without replacement; `grid` evaluates every combination.
Trial limit	6	Range 1–1000. A grid must fit entirely within the limit.
Sampling / inner-split seed	5209	Controls random candidate sampling and a newly needed inner holdout.
Inner validation fraction	0.20	Range 0.01–0.49; used when a new inner holdout is needed, such as full-training leave-one-out. Existing validation membership is otherwise retained.
Initial candidate fields	Learning rate and batch size	Learning rates 0.0001/0.001/0.01; batch sizes 16/32.

Searchable training fields are optimizer, learning rate, weight decay, batch size, epochs, patience, min delta, and momentum. Registered parameters of existing layers can also be searched. Each parameter has 1–100 distinct explicit candidates; a search has 1–12 parameter fields and at most 1,000,000 possible combinations. Search does not automatically vary preprocessing, data splits, or arbitrary unlisted training fields. It currently requires single-stage training.

Candidates are ranked by the **configured validation loss** (`mse`, `mae`, or `focal_mae`), on the training-fitted target scale. They are not ranked by outer-test Pearson correlation. An existing validation partition is used when present. Under default leave-one-out, search first creates an inner split from the N−1 outer-training individuals, then refits the selected candidate on all N−1 individuals for its selected epoch count. The held-out individual remains excluded throughout selection.

Expect up to the trial limit plus one formal fit per outer fold, multiplied by the number of separately trained traits. Invalid candidates are recorded and can be skipped; if none succeeds, the search fails. Read `search/trials.json`, individual trial records, and the selected-project/summary files to see actual settings and failure reasons. The UI's preflight count and progress units describe work, not a completion-time prediction.

10 Run checks, start training, and monitor progress

1. Save the project, inspect the actual split, and acknowledge leave-out workload if applicable.
2. Click **Check before running**. The check uses the actual input data, train-fitted preprocessing, every planned fold, and relevant search candidates. It checks shapes and performs bounded numerical forward/backward probes on the selected device. Probe weights are discarded.

3. Inspect the checklist and its report. Configuration, input content, code, dependencies, or the allocation budget changing can invalidate a successful check. Some candidate failures may be recorded while valid alternatives remain usable.
4. Click **Start training** after checks pass. The GUI runs one job at a time in a worker process.
5. In **Run monitor**, inspect the current trait/fold/trial, epoch, best validation epoch, learning curves, elapsed process time, and memory. **Training guide ?** explains common curve patterns using training/validation history; it does not alter parameters or inspect outer-test metrics.

A preflight success is evidence that bounded checks passed, not a guarantee that a complete large run will fit memory. Parameter/buffer storage is not total memory: activations, gradients, optimizer state, attention scores, reference graphs, and batch size also matter. The standard model allocation budget is 256 MiB for parameters plus registered buffers; several components additionally impose element-count limits. If checks fail, inspect the cited layer and reduce the relevant width, token count, batch size, or graph complexity before trying again.

Ordinary BatchNorm can fail when the final training batch contains only one usable observation per channel; changing batch size or normalization may resolve that specific error. For CUDA models containing AdaptiveMaxPool1d, the implementation normally disables strict deterministic mode; explicitly demanding its unsupported strict CUDA backward path causes an error. Review the effective settings saved with the run.

The monitor displays process RSS and PyTorch CUDA allocated/reserved memory, not whole-system memory or all GPU usage. Archived RSS peaks are sampled every 0.1 s and can miss a brief maximum; CUDA peaks cover the PyTorch allocator, excluding driver and other-process allocations. Overall progress counts planned work units, not elapsed-time percentage.

Click **Cancel current job** to request cancellation at a safe computation boundary. If the worker does not exit within 10 seconds, the GUI can force-stop it. Interrupted output retains an incomplete/failure state and is not a successful model. Cancellation is not a promise of checkpoint resume. Preserve completed results and start a new task when continuing; separate-trait batch retry can reuse verified completed child tasks under its own unchanged-input/code/environment checks.

11 Train several traits separately or jointly

Choose the task that matches the biological question. A batch of independent single-trait fits and one shared multi-output network produce different models and losses.

Separate trait tasks. Select multiple phenotype columns in the data interface, choose **Separate training (one model/trait)**, and click **Apply selected traits and task mode**. Each trait receives a separate model and result directory. A shared ID plan is projected onto each trait's usable labels, so missing phenotypes can produce different actual counts without silently reshuffling individuals for each trait. Tasks run serially; one failed trait is reported separately and does not automatically erase successful tasks. A separate-trait batch requires a single-output model and cannot also enable joint target heads. Applying this task mode clears single-trait reused split-file settings, because the batch manages its shared plan.

Joint outputs. In **Network builder → Inputs and outputs**, open **Single / multiple trait / environment heads**. Choose **New individuals, known environments**, add 2–64 targets, and fill in each target's ID, phenotype column, trait, environment, unit, correction description, and positive weight. Click **Apply heads**. The linear output width becomes the number of targets. The first target column is also the primary display trait.

Use a wide phenotype table: one individual per row, one trait/environment observation column per head. All heads of one individual stay in the same partition. Missing observations are masked in the loss; each head still needs training observations and, when a validation partition exists, validation observations. Per-target

means/scales use observed training labels only. The objective combines target-wise losses with the declared weights, so units, scaling, and weights need deliberate choices.

Target IDs and columns must be unique, and each trait/environment pair must identify a single head. Target columns must not also be prediction-input features. Auxiliary phenotypes are training labels, not required inputs for new-individual prediction. This protocol supports new individuals in the declared known environments; it does not provide extrapolation to an undeclared environment. Joint targets cannot currently be combined with staged/reconstruction objectives. The additional target table is limited to 20 million cells.

12 Optional staged training

Open **Configure pretrain / joint / frozen stages** only when the architecture and scientific question require it. A stage declares its mode, epoch limit, regression/reconstruction weights, and frozen layer IDs. Up to eight stages are supported, and their epoch limits replace the global maximum-epochs value.

Stage mode	Required weights	Purpose
pretrain	Regression 0; reconstruction > 0	Fit the declared reconstruction objective.
joint	Both weights > 0	Fit regression and reconstruction together.
regression	Regression > 0; reconstruction 0	Fit phenotype regression.

A reconstruction stage requires an explicit reconstruction head; the final stage must train regression. The generic Autoencoder template creates reconstruction pretraining followed by regression, with each stage initially using the current global epoch value; its regression stage freezes the encoder layers recorded by that template. Review these stages instead of assuming the global limit applies only once.

Every stage resets the optimizer, learning-rate scheduler, and early-stopping state. Validation-based stages restore their own best checkpoint; fixed-epoch stages keep their final checkpoint. Freezing affects both parameters and running statistics and follows actual shared-module ownership. Reconstruction loss uses observed genotype entries, not imputed values treated as known observations. Outer-test data do not train either objective. Hyperparameter search and joint multi-target output are not currently supported together with this staged workflow.

13 Train a final model for new individuals

After evaluating the chosen workflow, use **Training results → Train final model from current project** to create a deployment model. This action uses the **current project configuration**, not automatically the fold with the highest test score.

The final-model operation first makes a fresh internal train/validation selection split with no outer test. Its default validation fraction is 0.20, or the configured search fraction when search is enabled. It selects settings/epoch counts internally, then reinitializes the model and fits on all eligible labelled individuals for the selected number of epochs. Preprocessing and reference/kernel states are fitted again on those full training data. A staged model uses the selected epoch count for each stage.

The output separates internal selection from the final `model` and records its provenance. It has **no new independent test accuracy**. The earlier CV results describe the evaluated training process; they should not be relabelled as an independent score measured on the final model. Export the completed final model package for new-individual prediction, keeping the evaluation results and selection records as supporting evidence.

14 Read, export, and interpret results

Open **Training results**. For an existing run, click **Open another result…**, select its completed result directory, then **Load result**. Use the directory picker so that the field contains an absolute path; do not manually enter a relative model/result directory. The page separates **1. Data overview**, **2. Actual model and settings**, **3. Prediction results**, and **4. Save and apply**. Incomplete archives or altered checksummed artifacts are not accepted as completed results.

14.1 Select the correct evaluation scope

Choose **Trait / target**, **Evaluation source**, and **Partition** before interpreting a number or exporting a report.

Choice	Interpretation
Pooled OOF (one outer test per individual) with **Out-of-sample test / OOF**	Combines saved outer-test predictions. Each eligible individual contributes once per target in this CV run.
Fold N	Shows that fold's actual model, settings, and selected partition. Different folds may have selected different settings.
Validation (used for selection)	Validation informed model/epoch selection; its metric is not an independent final test.
Training fit (not out of sample)	Fit to training observations. In CV, select a single fold; repeated training appearances are not pooled.
Final model / internal selection (no independent test)	Final refit has no newly created independent test score. Consult the original evaluation experiment.

The page displays observed-versus-predicted and individual-error plots, learning curves when recorded, and fold Pearson plots for CV where applicable. **Training / search details** shows fitting and search records. Missing timing is shown as unavailable; shared multi-target time is not divided into invented per-trait runtimes.

Pearson r measures linear association and has no unit. RMSE and MAE measure prediction error in the input phenotype's units; RMSE is `sqrt(mean((predicted - observed)^2))`. **More metrics and definitions** includes Spearman correlation, R^2, bias, calibration, and improvement over a training-mean baseline. R^2 can be negative and is not Pearson r squared. Bias is mean(predicted − observed); a positive value indicates overprediction. Calibration fits observed = a + b × predicted as a diagnostic, without automatically correcting predictions.

Three summaries must be kept distinct:

1. **Pooled OOF Pearson** is calculated once from concatenated outer-test observations and predictions.
2. **Fold Pearson mean ± SD** is the unweighted mean and sample SD (`ddof=1`) of valid fold correlations. Undefined correlations are excluded, not replaced by zero; retain valid/total fold counts. Folds share training observations, so this SD is descriptive rather than a confidence interval. Leave-one-individual-out and leave-one-genotype-group-out use pooled OOF reporting rather than the ordinary fold Pearson summary.
3. **Model disagreement SD** in new-individual prediction is a population SD (`ddof=0`) across loaded members for one individual, in original target units. It is neither of the above summaries and is not a confidence interval or breeding-value reliability.

Constant observed values, constant predictions, or fewer than two observations make Pearson unavailable. Preserve the accompanying reason rather than substituting zero. A four-dataset manuscript table containing mean ± SD requires a separately defined aggregation of the exported fold records; this GUI does not silently combine independent seeds or repeated runs.

Predictions are inverse-transformed through GPNB's saved target standardization, if used, to the input training phenotype scale. Upstream log transformations, corrections, and units are not inferred or automatically reversed. If the input was log(yield), the returned scale remains that supplied log(yield) scale. A unit label is metadata, not a unit conversion. Phenotype prediction is not automatically an estimated breeding value.

14.2 Choose what to save

Under **4. Save and apply**, set **Save location** and an optional **Save name**. Each operation creates a new directory with a unique suffix.

Button	Content and purpose
Export results	A result report with metrics, per-sample records, figures, actual settings, and timing. **Report export scope** defaults to **All targets**; **Selected target** limits the report to the displayed target. Partition and fold selections are respected.
Save model	A frozen prediction package containing all declared members, tensor weights, architecture, and fitted preprocessing/reference states. Use it for new individuals.
Export full experiment	An immutable experiment copy with saved source, dependency records, and `reproduce.py`. **Include input data in full experiment** is unchecked initially; enable it when training inputs should accompany the retraining export.

Export results

is physically in this save section, even though it acts on the result plots above. A report normally contains `report.html`, top-level `metrics.csv`, and `target_001`, `target_002`, … directories. Each target directory contains `predictions.csv`, `summary.json`, `timing.csv`, applicable `fold_metrics.csv`, and figures. Default figure formats are PNG, SVG, and PDF; PNG output is **150 dpi**. Use the vector SVG/PDF for manuscript preparation or perform an explicitly specified publication export separately; the standard button does not create a 600-dpi raster figure. Keep the report folder together so the offline HTML can load its linked figures and CSV files.

For CV, `fold_metrics.csv` records outer-test metrics for every fold even when the selected report view is one fold or a training/validation partition. Its explicit `partition=test` column identifies that scope. Fold rows are not learning-curve points.

A prediction model package does not need the original training input files, and it does not copy training phenotype values or the original complete genotype files. It still contains identities; KNN imputation, kernel models, and individual-graph models can require saved reference genotypes. A package is therefore not anonymized. It requires a compatible GPNB/PyTorch installation and is not a standalone executable. Preserve the entire folder, including `complete.json`, `model_package.json`, and every member directory.

Full experiment exports support `reproduce.py verify`, `predict`, and, when training inputs were included, `train` in a compatible Python environment. They do not embed Python or PyTorch. Retraining checks recorded numerical package/Python versions and the original device; a cross-hardware guarantee of identical numerical results is not provided. Historical archives without a saved source snapshot cannot acquire one from the current installation. Keep the original archive intact.

Train final model from current project is a separate operation: it uses the current project for internal selection and then refits eligible training individuals. It does not simply choose the CV fold with the highest outer-test score. **More: compare CV results → Compare matching folds** compares compatible completed CV runs; do not substitute an unpaired set of different splits.

14.3 CSV field reference

CSV files are UTF-8. Import FID and IID as text in spreadsheet software to preserve leading zeros. Row-index fields such as `fam_index`, `training_bim_index`, and `new_bim_index` are zero-based; original CV fold numbers are one-based. A fixed-split training record uses `fold=0`.

Training run `predictions.csv`:

CSV

```
dataset_id,trait,fold,seed,model_sha256,partition,fam_index,FID,IID,observed,predicted
```

Joint multi-target training appends `target_id,environment,unit`. A CV root's `predictions.csv` contains OOF test rows only; each fold directory retains its own train/validation/test records. Missing multi-target observations can be blank while predictions remain present.

Report `metrics.csv`:

CSV

```
target_id,trait,unit,n,pearson,pearson_reason,rmse,mae,mse,spearman,spearman_reason,r2,r2_reason,bias,c
alibration_intercept,calibration_slope,calibration_reason,baseline_mse,baseline_improvement
```

`n` counts the finite observation/prediction pairs used for that view. Missing metrics remain empty with reasons when applicable.

Report `fold_metrics.csv`:

CSV

```
partition,fold,n,pearson,pearson_reason,rmse,mae
```

New-individual `predictions.csv`:

CSV

```
dataset_id,trait,fam_index,FID,IID,predicted,target_id,environment,unit,model_version,identity_status,m
odel_disagreement_sd,member_count,selection_rank
```

Optional phenotype scoring appends `observed,prediction_minus_observed,phenotype_match`. `selection_rank` is blank when input order is retained, otherwise it is a one-based within-target rank. Ties preserve input order. The CSV records a model version, while `report.json` records all member versions and model/weight/preprocessor hashes.

New-individual `marker_alignment.csv`:

CSV

```
training_bim_index,new_bim_index,marker_id,retained
```

It describes the first member; ensembles additionally write `marker_alignment_member_N.csv`. `retained` indicates that member's fitted marker mask.

New-individual `samples.csv`:

CSV

```
fam_index,FID,IID,missing_genotypes
```

The prediction directory also includes `report.json` (identity, scoring, input and member provenance), `environment.json`, `prediction_values.npy`, `model_disagreement.npy`, and the completion inventory. Do not edit the sealed CSV files in place; copy them for analysis.

15 Predict new individuals

1. Open **New individual prediction**, use **Browse…** beside **Model package or completed result** to select the absolute directory path, and click **Load and check model**. Do not manually enter a relative model directory. Alternatively, use **Predict with this model** → from a loaded result. Loading identifies the required targets, genotype contract, and any extra numeric tables.

2. Supply **Query BED**, **Markers BIM**, and **Samples FAM**, or configure **Numeric matrix (CSV/TSV/TXT)**. Provide a query table for every additional input branch. The software does not automatically reuse a training covariate file.
3. Enter **Query cohort name** (initially `new_cohort`) and the appropriate **Individual identity source**. The cohort name alone does not establish identity.
4. Optionally enable **Provide observed phenotypes for scoring** and supply a new phenotype file. Column conventions are suggested from the loaded model; training phenotype values are not reused. Multi-target scoring uses the original target column names.
5. Choose **Output root**, **Prediction device** (`cpu` initially), and **Candidate ranking**. The default is **Keep input order**. Click **Check and predict**.

All original training marker identities must be available, including markers subsequently removed by that member's fitted filter. Query marker order may differ from training order: GPNB aligns by marker ID, then requires all six stored marker fields to match, including chromosome, positions, and case-sensitive A1/A2. Extra query markers are ignored and counted in the report. The software does not silently flip alleles or complement strands. In matrix mode, columns must first match the query's own BIM; no-BIM inputs must retain the same coding-source declaration.

For an ensemble, **Equal average of all members (default)** predicts each member using its own frozen preprocessing and averages predictions after returning them to original target units. All declared members must be present and valid. **Advanced: use a single fold** selects an original fold explicitly; do not choose it from outer-test performance. A single member has zero between-member disagreement, which does not imply zero prediction error.

Identity controls affect scoring eligibility, not whether a phenotype-free prediction can be produced:

UI choice / recorded status	Meaning
Source relation unknown (identity uncertain) → `identity_uncertain`	Default. Matching ID text or genotype values alone does not establish individual identity.
Same ID source as training data	IDs are compared to the saved members' training, selection, and fixed-graph contexts. Statuses are `seen_training`, `seen_selection`, `seen_graph_context`, or `confirmed_unseen`.
Independently confirmed new individuals → `confirmed_unseen`	Requires a recorded verification basis, such as registry and collection records showing no training/selection overlap.

Exact matching genotype-and-ID source-file hashes can identify the original cohort and automatically establish the same-source relation; a contradictory “confirmed new” declaration is rejected. A fixed-cohort transductive graph cannot be repurposed for arbitrary new individuals. Seen, selection, graph-context, and uncertain records are reported separately; only finite observed values for `confirmed_unseen` individuals enter `new_individual_metrics`.

Without phenotypes, the output contains predictions and no observed-accuracy estimate. With phenotypes, missing or unmatched observations do not remove predictions: `phenotype_match` reports `matched`, `missing_trait`, or `unmatched_id`. Invalid matched observations cause an error that must be corrected or explicitly declared missing. Scoring is performed after frozen predictions; query phenotypes do not retrain a model, select members, or construct reference features. Ensemble predictions for the original training cohort are not OOF—use the saved OOF results to evaluate that cohort.

16 Troubleshooting

Start with the displayed error code and file/field name. Preserve the failed output directory: `failure.json`, `cancelled.json`, job logs or incomplete markers can explain what happened. Do not turn an incomplete run into a completed result by deleting its marker or editing checksums.

Symptom	Locate the cause and act
Setup stops during download	Read the latest `logs/setup-*.log` and `.status.json`; identify whether bootstrap or dependencies failed. Check connectivity to the named source and the free-space threshold. Rerun the same profile online. Do not use `-Offline` for a first installation.
Start reports an incomplete or changed environment	Run setup in this exact copy and profile. Moving the folder, changing dependency/configuration files, or mixing environments can invalidate the ready record. Use `-Repair` for a damaged installed environment.
GPU unavailable / `E_DEVICE`	Review the CUDA profile's `logs/doctor-*.json`, driver information and actual computation failure. Install a compatible profile or use CPU. Select CPU explicitly in the project and prediction page; the program does not silently switch devices.
Missing companion files / `E_FORMAT`	Re-select a matching BED/BIM/FAM set, or check every numeric-matrix option. Inspect delimiter, header, row/column layout, exact hard-call tokens, matrix coding and allele basis. Renaming a file extension does not convert its content.
No matched individuals / `E_DUPLICATE`	Compare the full ID strings in the data-check tables. Preserve leading zeros, remove accidental whitespace, choose the right ID column names and pair/IID mode, and resolve genuine duplicate keys at the data-source level. Do not fix matching by reordering phenotype rows.
Invalid target / `E_TRAIT`	Open `invalid_traits.csv` or the check report, locate the row/trait, and distinguish a typo from an intended missing value. Declare the exact missing token when appropriate. Do not replace missing phenotypes with zero as a workaround.
Start training is disabled / `E_PREFLIGHT`	Complete **Check before running** after the latest edits. Check invalid JSON fields, input files, shape errors, or a stale receipt. For leave-one-out, generate and acknowledge the real workload first.
Network error / `E_MODEL`, `E_PORT`	Read the indicated layer and expected tensor layout. Check convolution/pooling lengths, attention width/head divisibility, explicit branch ports and regression output size. A data-filtered marker count can differ from the original count. Use **Shapes and summary** and preflight; parameter storage is not total working memory.
Memory failure / `E_MODEL_LIMIT`, `E_REFERENCE`	Inspect the named storage/working-memory budget and any resource report. Smaller batches can reduce activation memory; a smaller model, fewer tokens, or an appropriate reference strategy may be required. Kernel matrices, KNN references and attention have their own bounds. Change the project deliberately and rerun checks; do not merely increase a limit without checking available memory.
Prediction marker error / `E_MARKER`	Compare `marker_alignment.csv` when produced and the reported marker. Supply all original training markers with consistent metadata/allele coding; reordered query markers can be aligned, but an allele flip is not repaired automatically. Review the declaration for a no-BIM matrix.
Model load fails / `E_BUNDLE`	Select the full completed experiment/model-package directory using **Browse…**, not a standalone weight file. Keep all members and state files. If a copied package fails checksum validation, obtain an intact copy; do not reseal it to hide missing or altered files.
Result path cannot be saved / `E_IO`, `E_EXISTS`	Check the named path and write permissions; choose a new output directory separate from inputs and existing experiments. The GUI appends unique suffixes; CLI `--output` directories must not already exist. Use an absolute location selected through the GUI for prediction/export paths.

Symptom	Locate the cause and act
Pearson is undefined or weak	Check the number of finite pairs, whether predictions or observations are constant, the evaluation partition, and the recorded reason. Undefined Pearson is not zero. A three-epoch synthetic demonstration is not an accuracy-optimized analysis. Do not tune the model against the held-out test result.
A CSV looks garbled or loses leading zeros in a spreadsheet	Import it explicitly as UTF-8 and set ID columns to text. Output CSVs use UTF-8 without a BOM. Preserve the original file; do not let a spreadsheet editor silently rewrite source IDs.
A cancelled run cannot be loaded as a model	Cancellation preserves incomplete output and is not checkpoint resume. Use a fresh output for a new ordinary fit. The separate-trait retry operation has its own compatibility checks and is not general training resume.

For a diagnostic report from an installed CPU profile:

PowerShell

```
.\Start_GPNB_v1.cmd -Profile cpu -Doctor -Language en_US
```

This runs health checks rather than opening the main editor. Application startup writes `logs/launch-*.log`; health reports use `logs/start-*.json`. Worker requests and protocol logs are under `jobs/<id>`. Scientific output paths are described above. When reporting a problem, provide the error code, relevant settings and a small synthetic reproducer; remove private paths, personal identifiers and real input data from logs before sharing.

17 Optional reproduce the small example with commands

These are **PowerShell commands executed in the application root after CPU setup**. Use the local Python executable below; no activation step is required. All output directories must be new. If you repeat the exercise, choose new names consistently, including the preflight receipt path. `--lang` is the application CLI option; `-Language` belongs to the setup/start wrappers.

PowerShell

```
.\runtime\envs\cpu\Scripts\python.exe -m gpnb inspect .\examples\quickstart\project.json --scan --
output .\runs\quickstart_audit --lang en_US
.\runtime\envs\cpu\Scripts\python.exe -m gpnb preview-split .\examples\quickstart\project.json --
output .\runs\quickstart_split --lang en_US
.\runtime\envs\cpu\Scripts\python.exe -m gpnb preflight .\examples\quickstart\project.json --
output .\runs\quickstart_preflight --lang en_US
.\runtime\envs\cpu\Scripts\python.exe -m gpnb train .\examples\quickstart\project.json --preflight-
report .\runs\quickstart_preflight\summary.json --output .\runs\quickstart_train --lang en_US
.\runtime\envs\cpu\Scripts\python.exe -m gpnb save-model .\runs\quickstart_train --
output .\runs\quickstart_model --lang en_US
.\runtime\envs\cpu\Scripts\python.exe -m gpnb export-plots .\runs\quickstart_train --
output .\runs\quickstart_report --lang en_US
.\runtime\envs\cpu\Scripts\python.exe -m gpnb predict .\runs\quickstart_model --
bed .\examples\quickstart\query\genotypes.bed --phenotype .\examples\quickstart\query\phenotype.csv --
identity-relation confirmed_new --identity-note "Independently simulated query individuals with
disjoint QRY identifiers." --dataset-id synthetic_query --output .\runs\quickstart_prediction --lang
en_US
.\runtime\envs\cpu\Scripts\python.exe -m gpnb export .\runs\quickstart_train --include-data --
output .\runs\quickstart_experiment --lang en_US
```

`--include-data` is appropriate here because all inputs are synthetic. For real experiments, decide explicitly whether the recipient should receive the inputs. See available commands and arguments with:

PowerShell

```
.\runtime\envs\cpu\Scripts\python.exe -m gpnb --help
.\runtime\envs\cpu\Scripts\python.exe -m gpnb predict --help
```

The actual subcommands are `save-model` and `export-plots`. There are no `model-export` or `result-export` commands. The GUI requires a valid preflight receipt; CLI `train` accepts an optional `--preflight-report`, and the example explicitly supplies it.

18 Network components and all registered parameters

The current pure package registers **89 components**. This reference lists every registered parameter and its insertion default, extracted from that package. A template or an existing project can override these values. Input dimensions, channels, and output sizes inferred from upstream layers are not all user-entered parameters.

The GUI filters choices according to the declared inputs and current insertion point. Open **Why are options unavailable?** to inspect a missing option. An accepted scalar range does not guarantee that a combination has compatible tensor shapes or fits memory; shape inspection and preflight perform additional checks.

18.1 Read the layouts and ports

Layout	Shape excluding the batch/individual axis	Meaning
Vector	features	One flat feature vector per individual.
Channels	channels × positions	Layout for 1D convolution and pooling.
Sequence	positions × width	Layout for recurrent, token and attention layers.
Graph	loci × width	Features attached to a configured locus graph.
Grid	channels × rows × columns	An explicit user grid.
Mask	positions	Validity information for sequence positions.
Categorical	markers	Genotype/missing categories for embedding.

`x` is a main input; `a`/`b` are two branch inputs. `mask`, `query_mask`, and `context_mask` require masks; a true mask entry means a masked/padding position, while false means a valid position. `gate` and `weight` require vectors. `FeatureAffine.condition` specifically requires a Vector although its main `x` input can be Vector or Sequence. For multi-input components, the main-layout column is a summary rather than a promise that every listed layout combination is legal.

Examples: ToChannels changes a marker Vector into one channel over markers. ToSequence makes one scalar feature at each marker position. SingleToken instead puts the entire vector into a single sequence position. These produce different models. Flatten retains the batch axis and combines the remaining dimensions. Reshape must preserve the number of elements per individual; Permute only reorders non-batch axes.

In the tables, each entry is `parameter = insertion default` followed by its allowed choices or scalar range. `true`/`false` are checkbox states. Bounds are inclusive unless explicitly stated. Integer-list bounds apply to each element. Extremely large upper bounds are validation bounds, not practical recommendations.

18.2 Parameter vocabulary

Parameter family	Interpretation
out_features / out_channels / hidden_size	Output width, convolution channel count, or internal learned width. Increasing them usually increases storage and computation.
bias / affine / elementwise_affine	Whether to learn the corresponding intercept or affine scale/shift parameters.
p / dropout	Training dropout probability; prediction uses evaluation mode.
kernel_size / kernel_h / kernel_w	Number of positions covered by a kernel in each direction.

Parameter family	Interpretation
stride / dilation / padding	Movement between windows, spacing within a kernel, and explicit boundary padding.
groups / num_groups	Channel grouping. Divisibility and minimum-size checks still apply.
eps	Numerical stabilizer for normalization; in GIN, eps instead weights the self-node contribution.
momentum	Running-statistic update parameter; KerasBatchNorm1d uses retention momentum and therefore differs from BatchNorm1d.
patch_size / embedding_dim / pad_tokens	Markers per token, token width, and extra padding tokens.
num_heads / heads	Number of attention heads; applicable widths must satisfy component-specific divisibility constraints.
max_score_elements / max_message_elements / max_elements	Explicit computation-size guards, measured in elements rather than bytes. Batch size contributes to the actual checked cost.
axis / axes / start / stop	Concatenation/slicing axis or layout permutation. Permute axes are zero-based over non-batch dimensions; Concat/Slice axis uses tensor axes starting at 1 after batch. Slice stop is exclusive.
mode / readout	How position-wise features become an aggregate or output representation.

18.3 Dense layers, activations and dropout

Linear learns a weighted combination of its input features. Activations transform those values. Dropout adds training-time regularization and is disabled during prediction.

Component	Input layouts and ports	All registered parameters and insertion defaults
`Linear`	Vector, Sequence, Graph Ports: `x`	`out_features = 32`; integer [1, 2147483647] `bias = true`
`Identity`	Vector, Channels, Sequence, Graph, Grid Ports: `x`	None
`ReLU`	Vector, Channels, Sequence, Graph, Grid Ports: `x`	None
`GELU`	Vector, Channels, Sequence, Graph, Grid Ports: `x`	None
`Tanh`	Vector, Channels, Sequence, Graph, Grid Ports: `x`	None
`Sigmoid`	Vector, Channels, Sequence, Graph, Grid Ports: `x`	None
`SiLU`	Vector, Channels, Sequence, Grid Ports: `x`	None

Component	Input layouts and ports	All registered parameters and insertion defaults
`ELU`	Vector, Channels, Sequence, Grid Ports: `x`	None
`SELU`	Vector, Channels, Sequence, Grid Ports: `x`	None
`Softplus`	Vector, Channels, Sequence, Grid Ports: `x`	None
`Softsign`	Vector, Channels, Sequence, Grid Ports: `x`	None
`LeakyReLU`	Vector, Channels, Sequence, Grid Ports: `x`	`negative_slope = 0.01`; [0, 1]
`PReLU`	Vector, Channels, Sequence Ports: `x`	`init = 0.25`; [0, 1]
`Dropout`	Vector, Channels, Sequence, Graph, Grid Ports: `x`	`p = 0.2`; [0, 1]
`AlphaDropout`	Vector, Channels, Sequence Ports: `x`	`p = 0.2`; [0, 1]
`Dropout1d`	Channels Ports: `x`	`p = 0.2`; [0, 1]

18.4 Layout, slicing and marker grouping

These components change how one individual's values are arranged. They never exchange the individual/batch axis with a feature axis. Marker groups and positions follow file order.

Component	Input layouts and ports	All registered parameters and insertion defaults
`ToChannels`	Vector Ports: `x`	None
`ToSequence`	Vector Ports: `x`	None
`SingleToken`	Vector Ports: `x`	None
`MarkerChunk`	Vector Ports: `x`	`parts = 5`; integer [1, 2147483647] `index = 0`; integer [0, 2147483647]
`Pad1d`	Channels Ports: `x`	`left = 0`; integer [0, 2147483647] `right = 0`; integer [0, 2147483647]
`Flatten`	Vector, Channels, Sequence, Grid Ports: `x`	None

Component	Input layouts and ports	All registered parameters and insertion defaults
`Reshape`	Vector, Channels, Sequence Ports: `x`	`shape = [1,12]`; each integer [1, 2147483647] `layout = "Channels"`; choices: "Vector", "Channels", "Sequence"
`Permute`	Channels, Sequence Ports: `x`	`axes = [1,0]`; each integer [0, 1] `layout = "Sequence"`; choices: "Channels", "Sequence"
`Slice`	Vector, Channels, Sequence Ports: `x`	`axis = 1`; integer [1, 2147483647] `start = 0`; integer [0, 2147483647] `stop = 1`; integer [1, 2147483647]

18.5 One-dimensional convolution and pooling

Convolutions extract patterns along the declared position order. Pooling reduces or summarizes those positions. LocallyConnected1d uses position-specific weights rather than sharing one convolution filter across all positions.

Component	Input layouts and ports	All registered parameters and insertion defaults
`Conv1d`	Channels Ports: `x`	`out_channels = 8`; integer [1, 2147483647] `kernel_size = 3`; integer [1, 2147483647] `stride = 1`; integer [1, 2147483647] `dilation = 1`; integer [1, 2147483647] `bias = true` `padding = 0`; integer [0, 2147483647] `groups = 1`; integer [1, 2147483647]
`DepthwiseSeparableConv1d`	Channels Ports: `x`	`out_channels = 8`; integer [1, 2147483647] `kernel_size = 3`; integer [1, 2147483647] `stride = 1`; integer [1, 2147483647] `dilation = 1`; integer [1, 2147483647] `bias = true` `padding = 0`; integer [0, 2147483647]
`CausalConv1d`	Channels Ports: `x`	`out_channels = 8`; integer [1, 2147483647] `kernel_size = 3`; integer [1, 2147483647] `stride = 1`; integer [1, 2147483647] `dilation = 1`; integer [1, 2147483647] `bias = true`
`LocallyConnected1d`	Channels Ports: `x`	`out_channels = 8`; integer [1, 2147483647] `kernel_size = 3`; integer [1, 2147483647] `stride = 1`; integer [1, 2147483647] `dilation = 1`; integer [1, 2147483647] `bias = true` `padding_left = 0`; integer [0, 2147483647] `padding_right = 0`; integer [0, 2147483647] `boundary = "file_order"`; choices: "file_order"
`MaxPool1d`	Channels Ports: `x`	`kernel_size = 2`; integer [1, 2147483647] `stride = 2`; integer [1, 2147483647] `padding = 0`; integer [0, 2147483647] `ceil_mode = false` `dilation = 1`; integer [1, 2147483647]

Component	Input layouts and ports	All registered parameters and insertion defaults
AvgPool1d	Channels Ports: x	kernel_size = 2; integer [1, 2147483647] stride = 2; integer [1, 2147483647] padding = 0; integer [0, 2147483647] ceil_mode = false count_include_pad = true
AdaptiveAvgPool1d	Channels Ports: x	output_size = 1; integer [1, 2147483647]
AdaptiveMaxPool1d	Channels Ports: x	output_size = 1; integer [1, 2147483647]
GlobalPool1d	Channels Ports: x	mode = "mean"; choices: "mean", "sum", "max"

18.6 Normalization

Normalization layers act inside the network. They are different from training-set genotype or phenotype standardization in Training settings. Their statistics and train/evaluation behavior depend on the component.

Component	Input layouts and ports	All registered parameters and insertion defaults
BatchNorm1d	Vector, Channels Ports: x	eps = 1e-05; [1e-12, 1.0] momentum = 0.1; [0, 1] affine = true track_running_stats = true
KerasBatchNorm1d	Vector, Channels Ports: x	eps = 0.001; [1e-12, 1.0] momentum = 0.99; [0, 1] affine = true
InstanceNorm1d	Channels Ports: x	eps = 1e-05; [1e-12, 1.0] momentum = 0.1; [0, 1] affine = true track_running_stats = false
GroupNorm	Channels Ports: x	num_groups = 1; integer [1, 2147483647] eps = 1e-05; [1e-12, 1.0] affine = true
LayerNorm	Vector, Channels, Sequence Ports: x	eps = 1e-05; [1e-12, 1.0] elementwise_affine = true
RMSNorm	Vector, Channels, Sequence Ports: x	eps = 1e-05; [1e-12, 1.0] elementwise_affine = true

18.7 Recurrent layers

A recurrent layer reads a Sequence. Select sequence to keep all positions or final_state to obtain one vector. Bidirectional processing concatenates two directions and therefore changes output width.

Component	Input layouts and ports	All registered parameters and insertion defaults

`RNN`	Sequence
	Ports: `x`
		`hidden_size = 16`; integer [1, 2147483647]
		`num_layers = 1`; integer [1, 2147483647]
		`bias = true`
		`bidirectional = false`
		`dropout = 0.0`; [0, 1]
		`readout = "sequence"`; choices: "sequence", "final_state"
		`nonlinearity = "tanh"`; choices: "tanh", "relu"

`LSTM`	Sequence
	Ports: `x`
		`hidden_size = 16`; integer [1, 2147483647]
		`num_layers = 1`; integer [1, 2147483647]
		`bias = true`
		`bidirectional = false`
		`dropout = 0.0`; [0, 1]
		`readout = "sequence"`; choices: "sequence", "final_state"

`GRU`	Sequence
	Ports: `x`
		`hidden_size = 16`; integer [1, 2147483647]
		`num_layers = 1`; integer [1, 2147483647]
		`bias = true`
		`bidirectional = false`
		`dropout = 0.0`; [0, 1]
		`readout = "sequence"`; choices: "sequence", "final_state"

18.8 Genotype embeddings, tokens and masks

GenotypeEmbedding embeds hard-call/missing categories. Tokenize projects marker patches into a sequence. Masks identify valid positions so padding is not silently treated as observed signal.

Component	Input layouts and ports	All registered parameters and insertion defaults
`GenotypeEmbedding`	Categorical Ports: `x`	`embedding_dim = 8`; integer [1, 2147483647]
`Tokenize`	Vector Ports: `x`	`patch_size = 4`; integer [1, 2147483647] `pad_tokens = 0`; integer [0, 2147483647] `embedding_dim = 16`; integer [1, 2147483647] `bias = true`
`TokenMask`	Vector Ports: `x`	`patch_size = 4`; integer [1, 2147483647] `pad_tokens = 0`; integer [0, 2147483647]
`SequenceMask`	Sequence Ports: `x`	`valid_length = -1`; integer [-1, 2147483647]
`PositionEncoding`	Sequence Ports: `x`	`mode = "sinusoidal"`; choices: "sinusoidal", "learned" `max_length = 4096`; integer [1, 2147483647]
`ClassToken`	Sequence Ports: `x`	None
`PrependMask`	Mask Ports: `x`	None
`MaskedReadout`	Sequence Ports: `x`, `mask`	`mode = "mean"`; choices: "mean", "sum", "max"
`SequenceReadout`	Sequence Ports: `x`	`mode = "mean"`; choices: "mean", "sum", "max", "first", "last"
`ApplyMask`	Sequence Ports: `x`, `mask`	None

18.9 Attention and sequence mixing

Attention components require the listed explicit mask ports. Start from a token/attention template when assembling these connections. The number of score elements can grow rapidly with sequence length and batch size.

Component	Input layouts and ports	All registered parameters and insertion defaults

`SelfAttention`	Sequence
	Ports: `x`, `mask`
		`num_heads = 4`; integer [1, 2147483647]
		`dropout = 0.0`; [0, 1]
		`bias = true`
		`causal = false`
		`max_score_elements = 8000000`; integer [1, 2147483647]

`ProjectedAttention`	Sequence Ports: `x`, `mask`	`hidden_size = 64`; integer [1, 2147483647] `scale_dim = 8`; integer [1, 2147483647] `dropout = 0.5`; [0, 1] `max_score_elements = 8000000`; integer [1, 2147483647]

`TransformerEncoder`	Sequence
	Ports: `x`, `mask`
		`num_heads = 4`; integer [1, 2147483647]
		`dropout = 0.0`; [0, 1]
		`bias = true`
		`causal = false`
		`max_score_elements = 8000000`; integer [1, 2147483647]
		`num_layers = 1`; integer [1, 2147483647]
		`dim_feedforward = 64`; integer [1, 2147483647]
		`norm_first = true`
		`eps = 1e-05`; [1e-12, 1.0]
		`activation = "gelu"`; choices: "gelu", "relu"

`CrossAttention`	Sequence Ports: `query`, `context`, `query_mask`, `context_mask`	`num_heads = 4`; integer [1, 2147483647] `dropout = 0.0`; [0, 1] `bias = true` `max_score_elements = 8000000`; integer [1, 2147483647]
`AttentionPooling`	Sequence Ports: `x`, `mask`	`bias = true`
`MLPMixer`	Sequence Ports: `x`, `mask`	`token_hidden = 16`; integer [1, 2147483647] `channel_hidden = 32`; integer [1, 2147483647] `dropout = 0.0`; [0, 1] `eps = 1e-05`; [1e-12, 1.0]

18.10 Branches, gates and higher-level blocks

Use explicit Branches connections for multiple ports. Add and Multiply do not invent projections or align mismatched shapes. DenseMoE combines multiple dense experts; its expert count is not a number of traits.

Component	Input layouts and ports	All registered parameters and insertion defaults
`Add`	Vector, Channels, Sequence Ports: `a`, `b`	None
`Multiply`	Vector, Channels, Sequence Ports: `a`, `b`	None

Component	Input layouts and ports	All registered parameters and insertion defaults
`Concat`	Vector, Channels, Sequence Ports: `a`, `b`	`axis = 1`; integer [1, 2147483647]
`GatedAdd`	Vector, Sequence Ports: `a`, `b`	`bias = true`
`GatedMLP`	Vector, Sequence Ports: `x`	`hidden_size = 32`; integer [1, 2147483647] `bias = true` `dropout = 0.0`; [0, 1]
`SqueezeExcite1d`	Channels Ports: `x`	`hidden_size = 4`; integer [1, 2147483647]
`TCNBlock`	Channels Ports: `x`	`hidden_size = 8`; integer [1, 2147483647] `kernel_size = 3`; integer [1, 2147483647] `dilation = 1`; integer [1, 2147483647] `bias = true` `dropout = 0.0`; [0, 1]
`DenseMoE`	Vector Ports: `x`	`num_experts = 3`; integer [1, 128] `hidden_size = 16`; integer [1, 2147483647] `out_features = 8`; integer [1, 2147483647] `dropout = 0.0`; [0, 1]
`ChannelScale`	Channels Ports: `x`, `gate`	None
`ScaleByWeight`	Vector Ports: `x`, `weight`	None
`Softmax`	Vector, Channels, Sequence Ports: `x`	None
`OneMinus`	Vector, Channels, Sequence Ports: `x`	None
`FeatureAffine`	Vector, Sequence Ports: `x`, `condition`	`bias = true`

18.11 Locus graphs and individual references

GraphInput attaches a configured locus topology; graph message-passing layers then operate on loci. ReferenceGraphSAGE instead uses a declared individual reference protocol. These are not interchangeable graph types.

Component	Input layouts and ports	All registered parameters and insertion defaults
`GraphInput`	Vector Ports: `x`	None
`GCN`	Graph Ports: `x`	`out_features = 16`; integer [1, 2147483647] `bias = true` `max_message_elements = 8000000`; integer [1, 2147483647]

Component	Input layouts and ports	All registered parameters and insertion defaults

`GraphSAGE`	Graph
	Ports: `x`
		`out_features = 16`; integer [1, 2147483647]
		`bias = true`
		`normalize = false`
		`max_message_elements = 8000000`; integer [1, 2147483647]

`GIN`	Graph Ports: `x`	`out_features = 16`; integer [1, 2147483647] `eps = 0.0`; [-10.0, 10.0] `train_eps = true` `max_message_elements = 8000000`; integer [1, 2147483647]

`GAT`	Graph
	Ports: `x`
		`out_features = 8`; integer [1, 2147483647]
		`heads = 2`; integer [1, 128]
		`concat = true`
		`bias = true`
		`negative_slope = 0.2`; [0.0, 1.0]
		`dropout = 0.0`; [0.0, 1.0]
		`max_message_elements = 8000000`; integer [1, 2147483647]

`GraphReadout`	Graph Ports: `x`	`mode = "mean"`; choices: "mean", "sum", "max", "attention"
`ReferenceGraphSAGE`	Vector Ports: `x`	`hidden_size = 16`; integer [1, 2147483647] `num_layers = 2`; integer [1, 8] `bias = true` `max_message_elements = 8000000`; integer [1, 2147483647]

18.12 Explicit two-dimensional grids

ToGrid makes a user-specified grid from file-ordered features. The grid is an arrangement for computation, not an automatically reconstructed chromosome map or biological image. Review the padding and dimensions.

Component	Input layouts and ports	All registered parameters and insertion defaults
`ToGrid`	Vector Ports: `x`	`channels = 1`; integer [1, 2147483647] `rows = 0`; integer [0, 2147483647] `columns = 8`; integer [1, 2147483647] `layout = "file_order_user_grid"`; choices: "file_order_user_grid" `padding = "zero_end"`; choices: "error", "zero_end" `max_elements = 8000000`; integer [1, 2147483647]
`Conv2d`	Grid Ports: `x`	`kernel_h = 3`; integer [1, 2147483647] `kernel_w = 3`; integer [1, 2147483647] `stride_h = 1`; integer [1, 2147483647] `stride_w = 1`; integer [1, 2147483647] `padding_h = 0`; integer [0, 2147483647] `padding_w = 0`; integer [0, 2147483647] `out_channels = 8`; integer [1, 2147483647] `dilation_h = 1`; integer [1, 2147483647] `dilation_w = 1`; integer [1, 2147483647] `groups = 1`; integer [1, 2147483647] `bias = true` `max_elements = 8000000`; integer [1, 2147483647]

Component	Input layouts and ports	All registered parameters and insertion defaults
`MaxPool2d`	Grid Ports: `x`	`kernel_h = 3`; integer [1, 2147483647] `kernel_w = 3`; integer [1, 2147483647] `stride_h = 1`; integer [1, 2147483647] `stride_w = 1`; integer [1, 2147483647] `padding_h = 0`; integer [0, 2147483647] `padding_w = 0`; integer [0, 2147483647] `ceil_mode = false` `max_elements = 8000000`; integer [1, 2147483647]

`AvgPool2d`	Grid
	Ports: `x`
		`kernel_h = 3`; integer [1, 2147483647]
		`kernel_w = 3`; integer [1, 2147483647]
		`stride_h = 1`; integer [1, 2147483647]
		`stride_w = 1`; integer [1, 2147483647]
		`padding_h = 0`; integer [0, 2147483647]
		`padding_w = 0`; integer [0, 2147483647]
		`ceil_mode = false`
		`max_elements = 8000000`; integer [1, 2147483647]
		`count_include_pad = true`

`AdaptiveAvgPool2d`	Grid Ports: `x`	`rows = 2`; integer [1, 2147483647] `columns = 2`; integer [1, 2147483647] `max_elements = 8000000`; integer [1, 2147483647]
`BatchNorm2d`	Grid Ports: `x`	`eps = 1e-05`; [1e-12, 1.0] `momentum = 0.1`; [0.0, 1.0] `affine = true`
`GridReadout`	Grid Ports: `x`	`mode = "mean"`; choices: "mean", "sum", "max"

18.13 Autocorrelation and decomposition

These components implement the explicit operations used by the GPformer adaptation. Position order is the input order. AutoCorrelation has no padding-mask port; its positions must represent real input positions.

Component	Input layouts and ports	All registered parameters and insertion defaults
`AutoCorrelation`	Sequence Ports: `x`	`num_heads = 4`; integer [1, 2147483647] `factor = 2.0`; [0.1, 16.0] `bias = true` `max_elements = 8000000`; integer [1, 2147483647]
`SeriesDecomposition`	Sequence Ports: `x`	`kernel_size = 25`; integer [1, 1025] `output = "residual"`; choices: "residual", "trend"

18.14 Auxiliary reconstruction

ReconstructMarkers produces the declared reconstruction output. It requires explicit staged objectives; adding a reconstruction branch alone does not create an automatically weighted training objective.

Component	Input layouts and ports	All registered parameters and insertion defaults
`ReconstructMarkers`	Vector Ports: `x`	`bias = true`

18.15 Additional shape and interpretation rules

- `MarkerChunk` requires `index < parts <= retained marker count`. Remainders go to earlier chunks; boundaries are not automatically chromosomes or LD blocks.
- `Linear` acts on the last dimension. It does not directly accept Channels; convert the layout or flatten explicitly.
- Convolution groups must divide both inferred input and output channels. Kernel, dilation, stride and padding must leave at least one output position. Ordinary 1D pooling padding cannot exceed half its kernel.
- `BatchNorm1d` requires `track_running_stats=true` for saved, batch-independent prediction. `KerasBatchNorm1d` uses population moving variance and retention momentum 0.99; it is a separate component, not merely BatchNorm1d with the same numeric momentum. For ordinary BatchNorm1d/2d, preflight also checks the last training batch.
- `InstanceNorm1d` needs at least two spatial positions; GroupNorm requires channels divisible by its group count and at least two elements per group per individual. LayerNorm/RMSNorm normalize the last dimension in this implementation.
- A single-layer RNN/LSTM/GRU requires `dropout=0`, because the recurrent dropout setting acts between stacked layers. Use a separate Dropout component if that is the intended operation.
- Tokenize rounds up the marker count to whole patches and adds the specified padding tokens. TokenMask must use corresponding patch/padding settings. `SequenceMask.valid_length=-1` declares all sequence positions valid. PositionEncoding requires length no greater than `max_length`; ClassToken requires a corresponding Prepended mask when a masked downstream path is used.
- SelfAttention/Transformer/CrossAttention require the appropriate embedding width to be divisible by head count. ProjectedAttention is a separate projection-based operation; `scale_dim` is not a multihead count. With one input token, its attention matrix is 1 × 1.
- Add/Multiply require compatible shapes. Concat allows a declared non-batch axis and checks all other dimensions. Shared calls must satisfy module and shape compatibility; copying a layer does not share its weights.
- `ToGrid.rows=0` infers enough rows for the feature count under the declared channels/columns. `zero_end` pads at the end. Grid layout is `file_order_user_grid`, not a biological position map.
- AutoCorrelation uses per-individual circular FFT correlation and lag aggregation; no padding-mask input is provided. Its width must be divisible by `num_heads`. SeriesDecomposition requires an odd `kernel_size`; `trend` returns the endpoint-padded moving mean and `residual` subtracts it.
- Graph components need a configured, compatible graph/reference state. A small parameter count does not imply cheap message passing. Attention, graph and grid limits are operational guards, not recommendations to raise them until a run starts.
- AdaptiveMaxPool1d has a strict-deterministic CUDA backward limitation. The normal training path disables strict determinism for that CUDA pooling model unless strict behavior was explicitly demanded; review effective settings and the preflight report.
- A single-trait prediction head must yield one linear output per individual. One explicit output Dropout is supported; arbitrary post-head operations are not accepted. Multi-target output width must match its declared targets.

The 89-component count describes registered software components, not 89 biological prediction methods or 89 independently validated paper models. A supported component can still be unsuitable for a particular dataset, layout, task, or resource budget.